Grundlagen der IT – Klausurzusammenfassung

DHBW Heilbronn · Wirtschaftsinformatik · 1. Semester · Dozent: Dipl.-Inform. Gisbert Loff · Stand: 10.11.2026

Klausur auf einen Blick

Termin	Montag, 07.12.2026 (ab heute noch knapp 4 Wochen)
Dauer	48 Minuten, ca. 48–52 Punkte → **etwa 1 Punkt pro Minute**
Formales	Matrikelnummer auf Deckblatt und „Matelbogen“, **nicht den Namen**. Deutlich schreiben („Ich kann nur Punkte finden, wenn ich es lesen kann“). Antwortfelder im Klausurbogen nutzen.
Stoff	Folien VL01–VL05 und die Rückschauen; kein neuer Stoff im letzten Termin (21.10.). Punkteverteilung, MC-Regeln und Operatoren: nächste Seite.

Inhalt

0	Was in der Klausur drankommt – Punkte, Zeit, MC-Regeln, Operatoren, Lernreihenfolge	48 Min	**2**
1	Information und Digitalisierung	3 P	**3**
2	IT-Infrastruktur und Cloud-Computing	4 P	**5**
3	Mooresches Gesetz	2 P	**7**
4	Zahlendarstellung und Rechnen – Rezepte: Umrechnen, Zweierkomplement, Addition/Überlauf	3 P	**8**
5	Wahrheitstabellen, Boolesche Ausdrücke, Logikgatter – Rezepte: DNF, Vereinfachen, Gatter-Analyse	4 P	**12**
6	Programmierung	2 P	**16**
7	Datenstrukturen – Rezept: Stack-/Queue-Traces	6–8 P	**18**
8	Suchen und Sortieren – Rezept: Sortier-Durchläufe	6 P	**22**
9	Rechnerarchitektur: Von Neumann vs. Harvard	8 P	**25**
10	Speicher: Arten, Hierarchie, RAID	4 P	**28**
11	Datenbanken und SQL – Rezept: SELECT, WHERE, JOIN, GROUP BY	4–6 P	**30**
★	Spickzettel – alles Wichtige auf einer Seite für den Vorabend		**34**

So ist jedes Kapitel aufgebaut

1. **„In einem Satz / Stell dir vor“** (abgerundeter Kasten): der Kern des Kapitels in einem Satz und ein Bild aus dem Alltag, das du dir merken kannst.
2. **Stoff** mit Tabellen und Skizzen, wie auf den Folien.
3. ▶ **Rezept** (Kasten mit dicker Linie links): Schritt-für-Schritt-Anleitung für jede Rechen- und Anwendungsaufgabe, jeweils mit vollständig durchgerechnetem Beispiel.
4. **Typische Klausurfallen** und **Das musst du können** zum Abhaken.

Legende. Grundlage sind die Folien VL01–VL05; Vorrang haben die **Rückschauen** (dort steht, was tatsächlich behandelt wurde). (Rückschau) = in einer Rückschau, also sicher behandelt. (nur Folie) = im Foliensatz, aber nicht in der Rückschau. *(Ergänzung)* = steht nicht auf den Folien, nur zum Verständnis, nicht als Klausurstoff verlassen. Kästen: **abgerundet** = In einem Satz / Stell dir vor, **dicke Linie links** = Rezept, **gestrichelt** = typische Klausurfallen, **grau hinterlegt** = Checkliste „Das musst du können“, **Doppellinie links** = Folienfehler. Alle Rechenbeispiele wurden mit Python nachgerechnet, alle SQL-Ergebnisse mit SQLite.

Was in der Klausur drankommt — 48 Min

Offizielle **exemplarische Punkteverteilung** aus der Klausurvorbereitung (VL05 F. 8–9). Faustregel: **1 Punkt ≈ 1 Minute**. Die Spalte „Zeit" ist dein Zeitbudget pro Aufgabe; wer länger braucht, geht weiter und kommt am Ende zurück. Kästchen abhaken, wenn das Thema sitzt.

✓	Nr.	Thema (Wortlaut VL05)	Punkte	Zeit	Was du tun musst	Kap.
☐	1	Information und Digitalisierung	3	3 Min	nennen, kurz erläutern, erläutern	1
☐	2	IT-Infrastruktur und Cloud-Computing (Komponenten im Unternehmen, Struktur der IT-Landschaft)	4	4 Min	7 Komponenten, Client-Server, Cloud	2
☐	3	Mooresches Gesetz	2	2 Min	MC wortgenau prüfen (Beispiel aus VL05)	3
☐	4	Zahlendarstellung und Rechnen („sieht oft einfacher aus als es ist")	3	3 Min	2/8/10/16, Zweierkomplement, Kommazahlen	4
☐	5	Wahrheitstabelle, Boolesche Ausdrücke, Logikgatter (Analyse und Synthese)	4	4 Min	Text → Tabelle → DNF → Schaltung und zurück	5
☐	6	Programmierung (allgemeine Fragen)	2	2 Min	Begriffe abgrenzen, Rekursion	6
☐	7	**Datenstrukturen** (Bedeutung, Funktionsweise, APIs wie push/pop/enqueue/dequeue)	**6–8**	6–8 Min	Operationsfolgen durchspielen, Binärbaum bauen/löschen	7
☐	8	**Suchen und Sortieren** (Eigenschaften der Verfahren)	**6**	6 Min	Voraussetzung, Ablauf, Aufwand O(…)	8
☐	9	**Rechnerarchitektur**: Von-Neumann vs. Harvard (Skizzen, Eigenschaften, Pro/Contra)	**8**	8 Min	beide Skizzen zeichnen, Zyklus, Pro/Contra	9
☐	10	Speicher (Hierarchie, RAID)	4	4 Min	Pyramide, Cache, RAID-Level, Kapazität	10
☐	11	Datenbanken (Eigenschaften, SQL analysieren und schreiben)	4–6	4–6 Min	Begriffe, ER-Modell, SQL ausführen/schreiben	11
		Summe	**ca. 48–52**	**48 Min**	**Dickste Brocken: 9 (8 P), 7 (6–8 P), 8 (6 P) = 20–22 P, fast die Hälfte**	

MULTIPLE CHOICE – REGELN (VL05 F. 4)

- Alle richtigen Antworten ankreuzen. **Mindestens eine** ist immer richtig.
- Jede richtig angekreuzte **und** jede richtig *nicht* angekreuzte Option: **+½ Punkt**.
- Jede falsch angekreuzte **und** jede falsch *nicht* angekreuzte Option: **−½ Punkt**.
- Negative Summe → 0 Punkte. **Gar nichts angekreuzt** → „nicht bearbeitet" → 0 Punkte.
- Folge: Bei 4 Optionen bringt jede sicher beurteilte Option +½. Raten lohnt nur, wenn du mehr als die Hälfte der Optionen sicher einschätzen kannst.

Operator (VL05 F. 7)	Was wird erwartet?
Nennen	Stichpunktartige Aufzählung der Begriffe, keine Erklärung
Kurz erläutern	1–2 (ggf. stichpunktartige) Sätze
Erläutern	Mehrere, i. d. R. **deutlich mehr als 2**, vollständige Sätze
Skizzieren	Zeichnung (Diagramm, Abbildung), sauber beschriftet

Rechnen und Logik sind laut Dozent „**DAS IST ÜBUNGSSACHE!!**" und „AUCH ÜBUNGSSACHE!!" – sauber schreiben, Rechenweg zeigen.

Empfohlene Lernreihenfolge bis zum 07.12. *(Empfehlung, Zeiten grob geschätzt)*

#	Was – und warum	Lernzeit
1	**Rechnerarchitektur** (Kap. 9): beide Skizzen aus dem Kopf, 5 Komponenten, 5-Phasen-Zyklus, Pro/Contra – *größter Einzelblock, Skizze ist planbar*	2–3 h, Skizze 3× zeichnen
2	**Datenstrukturen** (Kap. 7): Stack/Queue/Liste/Baum mit Operationen, Binärbaum aufbauen und löschen – *Traces sind reines Üben*	2–3 h
3	**Suchen und Sortieren** (Kap. 8): Verfahren mit O(…), Durchläufe – *1 bis 3 zusammen 20–22 P, fast die Hälfte*	2 h
4	**Rechnen** (Kap. 4, 5): 2/8/10/16, Zweierkomplement mit fester Bitbreite inkl. Überlauf, Tabelle → DNF → Schaltung (IEC) – *„sieht oft einfacher aus als es ist"*	täglich 15–20 Min
5	**SQL** (Kap. 11): Abfragen von Hand ausführen und schreiben – *Rezept ist immer gleich*	1,5 h
6	**Begriffe** (Kap. 1, 2, 3, 6, 10, 11): Daten/Information, Digitalisierung, 7 Komponenten, Epochen, Cloud, Moore, Speicherhierarchie, RAID, DBMS, OLTP/OLAP – *schnelle Punkte durch Nennen und MC*	Karteikarten, 10 Min/Tag
7	**Nur kennen:** Gleitkomma (IEEE 754), Shift, Compiler-Phasen, Linker/Lader, Kodierungen, KI-Hardware – *Exkurse, nicht in den Rückschauen*	einmal lesen
8	**Generalprobe:** eine Probeklausur in 48 Min mit Uhr; am Vorabend nur noch den Spickzettel (letzte Seite)	1 h

IN DER KLAUSUR: ZEIT EINTEILEN

Erst 1 Minute alle Aufgaben überfliegen. Dann die sicheren Punkte (MC, Nennen, Skizzen), danach Rechnen. Pro Aufgabe höchstens so viele Minuten wie Punkte; hängst du fest, Rechenweg stehen lassen und weiter. Die letzten 3–4 Minuten: Matrikelnummer, Bitbreiten, Vorzeichen und MC-Kreuze kontrollieren.

1 · Information und Digitalisierung 3 P

IN EINEM SATZ Daten sind nur Zeichen; erst ein Kontext macht daraus eine Information, und Digitalisierung heißt, Analoges in solche Zeichen (0 und 1) zu übersetzen.

STELL DIR VOR Auf deinem Handy steht „7“. Sind das 7 °C, 7 °F oder 7 neue Nachrichten? Ohne Kontext weißt du es nicht. Und „digital“ ist wie eine Wetterstation, die jede volle Stunde einen gerundeten Wert notiert, statt eines Quecksilberthermometers, das ständig jeden Wert zeigt: **Treppenstufen statt Rampe** (zeit- und wertediskret).

Daten, Information, Wissen, Handlung (Rückschau)

Daten sind Rohdaten, also unverarbeitete Fakten wie Zahlen, Texte oder Symbole. Für sich allein haben sie keine Bedeutung. Erst wenn ein **Kontext** dazukommt, werden aus Daten **Informationen**: verarbeitete Daten mit Bedeutung, die Entscheidungen unterstützen. Informationen verdichten sich zu **Wissen**, und Wissen führt zu **Handlung**. Dieselben Daten können je nach Kontext eine völlig andere Information sein, wie das Folienbeispiel zeigt:

Daten	Kontext	Information	Wissen	Handlung
`000101010 → 42`	Kontostand in Euro	Ich habe 42 €	Kinokarte kostet 14 €	Ich gehe ins Kino
`000000111 → 7`	Temperatur °C (DE)	Es ist 7 Grad	Es ist kalt	Jacke anziehen
`000000111 → 7`	Temperatur °C (Kanada)	Es ist 7 Grad	Es ist warm	kurze Hose
`000000111 → 7`	Temperatur °F (USA)	Es ist 7 Grad	sehr kalt (≈ −13 °C)	Wintersachen

Verwandt (VL02, (nur Folie)): Information hat drei Aspekte: **syntaktisch** (Form, auswertbar), **semantisch** (Bedeutung durch Kontext), **pragmatisch** (was tue ich damit?). Computer arbeiten nach dem **EVA-Prinzip** (Eingabe – Verarbeitung – Ausgabe). Rechner arbeiten **binär**: 0 = keine Spannung, 1 = Spannung. **1 Bit** ist die kleinste Informationseinheit, **1 Byte = 8 Bit**; auf den Folien gilt 1 KB = 1024 Byte, 1 MB = 1024 KB usw.

Informationstheorie nach Shannon (1948, VL01, (nur Folie)): Modell Quelle → Sender → Kanal → Empfänger → Ziel; das Bit als Grundeinheit; **Entropie** = mittlerer Informationsgehalt; **Redundanz** entfernen = Kompression (z. B. RLE `AAAABBBCC → 4A3B2C`, Huffman), Redundanz hinzufügen = Fehlererkennung/-korrektur (Paritätsbit, Hamming-Code).

Digitalisierung – drei Sichtweisen (Rückschau)

Sichtweise	Kernaussage
Betriebswirtschaftlich	Verwendung von Daten und algorithmischen Systemen für **neue oder verbesserte Prozesse, Produkte und Geschäftsmodelle** (BMWK). Analoge Leistungserbringung wird ganz oder teilweise durch Leistungserbringung in einem digitalen, computerhandhabbaren Modell ersetzt. Querschnittstechnologie in praktisch allen Sektoren.
Gesellschaftlich	Übertragung der Lebens- und Arbeitswelten des Menschen auf eine digitale Ebene; heute oft im Kontext umwälzender ökonomischer Innovationen („**Disruption**“).
Technisch	Begriff aus der Elektrotechnik/Digitaltechnik: Überführung **analoger** in **zeit- und wertediskrete** (insbesondere binäre) Signale, um sie elektronisch zu übertragen, zu speichern und zu verarbeiten. Texte, Bilder, Töne, Filme werden als Folgen von 0 und 1 dargestellt.

Phasen der Digitalisierung (wirtschaftliche Sicht, nach Österle/Fleisch/Alt 2000) (Rückschau)

Phase	Was wird digitalisiert?	Beispiel
1	einzelne **Funktionen**	automatisierte Erstellung von Rechnungen
2	ganze **Funktionsbereiche** bzw. Organisationseinheiten	Rechnungswesen
3	**integrierte Prozesse** über Grenzen der Aufbauorganisation hinweg	Auftragsbearbeitung
4	individuelle **1:1-Integration** überbetrieblicher Prozesse	lieferantengeführte Bestände
heute	durchgängige **m:n-Integration** überbetrieblicher Prozesse bis zum Endkunden über eine globale Infrastruktur	B2B-/B2C-Marktplätze, Produktion on demand, Availability to promise, Long-Tail- und Digital-only-Produkte

Rolle und Nutzen der IT im Unternehmen (nur Folie)

- **Früher** (vor dem Internet): IT als untergeordnete „EDV-Abteilung“ für Effizienz in Routineabläufen, nicht strategisch (Carr 2003: „IT doesn’t matter“). **Heute**: IT ist Teil des Kerngeschäfts (Online-Vertrieb, CRM, digitale Produkte wie Spotify, Cloud-Betriebsmodelle, Data Analytics und KI).
- Nutzen in drei Stufen: **Automatisierung** (Abläufe „in IT gegossen“, schneller und standardisiert) → **Informatisierung** (die entstehenden Daten unterstützen Managemententscheidungen: BI, Data Warehouse) → **Transformation** (völlig neue Geschäftsprozesse, Business Process Reengineering).
- Ökonomische Effekte: verschobenes Verhältnis Fix-/variable Kosten, Netzwerkeffekte, Plattformökonomie (Uber, Airbnb), Long Tail, Freemium, sinkende Eintrittsbarrieren, Konvergenz von Branchen.

⚠ TYPISCHE KLAUSURFALLEN

- **Daten ≠ Information**: Daten haben ohne Kontext keine Bedeutung. Das 7-Grad-Beispiel zeigt: gleiche Daten, verschiedene Informationen.
- **Phasen der Digitalisierung** (wirtschaftlich, 1 → m:n-Integration) nicht mit den **Epochen der IT-Infrastruktur** (technisch, Großrechner → Cloud, Block 2) verwechseln. Die Folie warnt ausdrücklich davor.
- Technische Sicht: **zeit- und wertediskret**, nicht „zeitkontinuierlich".
- „Erläutern" heißt mehr als 2 ganze Sätze. Bei „Nennen" reichen Stichpunkte, Erklärungen kosten nur Zeit.

DAS MUSST DU KÖNNEN

- ☐ Daten/Information/Wissen/Handlung mit Beispiel erklären
- ☐ die drei Sichtweisen der Digitalisierung nennen und kurz erläutern
- ☐ die Phasen der Digitalisierung in der richtigen Reihenfolge nennen
- ☐ Bit, Byte, binär, EVA erklären
- ☐ Automatisierung – Informatisierung – Transformation unterscheiden
- ☐ Redundanz: Kompression vs. Fehlerkorrektur

2 · IT-Infrastruktur und Cloud-Computing 4 P

IN EINEM SATZ Der Client fragt, der Server antwortet; Cloud heißt, Rechner und Programme zu mieten, statt sie selbst zu kaufen und zu betreiben.

STELL DIR VOR Ein **Restaurant**: Du (Client) bestellst, die Küche (Server) kocht für alle. Bei N-Tier nimmt ein Kellner (Webserver) die Bestellung an, der Koch (Anwendungsserver) kocht, das Lager (Datenbankserver) liefert. Cloud ist **Strom aus der Steckdose** statt eigenem Kraftwerk. IaaS = gemietete Küche, PaaS = Küche mit Fertigteig, SaaS = Pizzeria.

Die Klausuraufgabe fragt laut VL05: „Welche Komponenten können Sie in einem Unternehmen vorfinden?“ und „Wie kann die IT-Landschaft strukturiert werden?“. Dafür brauchst du die sieben Komponenten, die fünf Epochen und die Client-Server- bzw. Cloud-Idee.

Sieben Hauptkomponenten einer modernen IT-Infrastruktur Rückschau

1. Hardwareplattformen
2. Betriebssystem-Plattformen
3. Datenverwaltung und -speicherung
4. Netzwerke und Telekommunikation
5. Anwendungssysteme
6. Internet (als Plattform)
7. Beratungsdienste und Systemintegratoren

> Merkhilfe *(Ergänzung)*: von unten nach oben denken – **Hardware → Betriebssystem → Daten → Netz → Anwendungen → Internet → Menschen, die alles zusammenbauen** (Berater, Integratoren). Beispiele: Server/PCs; Windows/Linux; Datenbank-Server, Speichersysteme; LAN/WLAN/Router; ERP (SAP), CRM; Webdienste; Systemhäuser.

Fünf Epochen der IT-Infrastruktur (technische Sicht) nur Folie (Epoche 4 und 5 in Rückschau)

Die Epochen überlagern sich: Technik einer Epoche lebt in späteren weiter (deshalb steht überall „– heute“). Treiber sind schnellere, kleinere, billigere Hardware sowie neue Architekturen und Algorithmen.

Epoche	Zeit	Kennzeichen	Beispiele
1. Groß- und Minirechner	1959 – heute	Großrechner bedient Tausende Terminals; alles von einem Anbieter. Minicomputer machen EDV günstiger und dezentraler.	IBM 1401/7090; DEC PDP-11, VAX
2. Personal Computing	1981 – heute	Arbeitsplatzrechner, „Wintel“-Standard, Tabellenkalkulation und Textverarbeitung; erst ab ca. 1990 vernetzt.	IBM-PC; Novell NetWare, Ethernet
3. Client-Server	1983 – heute	Clients (Eingabe) mit leistungsstarken Servern (gemeinsame Daten, Webseiten, Netzverwaltung) vernetzt; Rechenlast auf viele günstige Maschinen verteilt; 2-Tier/N-Tier.	SAP R/3 mit Internet Transaction Server + Datenbankserver
4. Betriebliche und internetbasierte Informationssysteme	1992 – heute	Netzwerkstandards integrieren unternehmensweite Anwendungen; ab 1995 TCP/IP verbindet heterogene Netze; verteilte Anwendungen (heute REST-APIs), Load Balancing; Kunden und Lieferanten verbinden sich über das Internet.	Supply Chain, Shops mit Lagerzugriff, Marktplätze
5. Cloud Computing	2000 – heute	Lösungen werden von externen Anbietern über das Internet bezogen (IaaS, PaaS, SaaS), meist im Abo, Zugang über den Browser; flexible virtuelle Instanzen.	AWS, Azure, Google Cloud, STACKIT; Salesforce (SaaS)

Client-Server und mehrschichtige Architektur (N-Tier)

Beim **Client-Server-Computing** ist der **Client** (Desktop, Laptop, Browser) die Eingabestelle des Benutzers. Der **Server** verarbeitet und speichert gemeinsam genutzte Daten, liefert Webseiten oder verwaltet Netzwerkaktivitäten. Bei **2-Tier** spricht der Client direkt mit einem Server. Die meisten Unternehmen haben **mehrschichtige (N-Tier-)Architekturen**: Die Arbeit wird über mehrere Server-Ebenen verteilt, z. B. Webserver → Anwendungsserver → Datenbankserver. Das Internet dient dabei „nur“ als technischer Zugang zum Web-/Anwendungsserver.

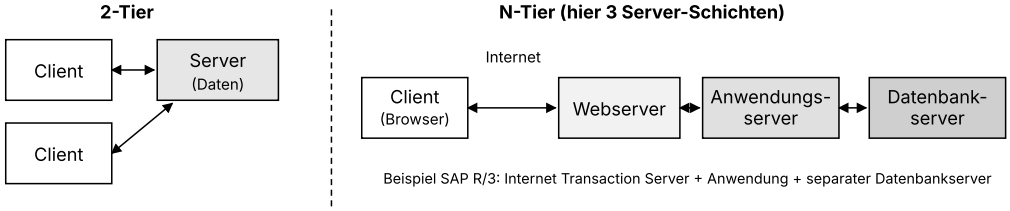


Abb. 2.1 – 2-Tier vs. N-Tier (eigene Skizze nach VL01)

Cloud Computing (Rückschau) (Vertiefung im 2. Semester)

> **Definition (Folie):** Vormals im Unternehmen betriebene Lösungen werden von externen Anbietern **über das Internet bezogen** (IaaS, PaaS, SaaS). Programme und Daten liegen nicht mehr auf lokalen Ressourcen des Unternehmens, sondern werden über das Internet bezogen, **meist als Abo**; der Zugang erfolgt i. d. R. über den **Browser**.

Modell	Was mietest du? *(Erklärung = Ergänzung)*	Beispiel *(Ergänzung)*
IaaS – Infrastructure as a Service	Rechenleistung, Speicher, Netz als virtuelle Maschinen; Betriebssystem und Anwendungen verwaltest du selbst.	virtuelle Server bei AWS EC2, STACKIT
PaaS – Platform as a Service	zusätzlich Laufzeitumgebung, Datenbank, Entwicklungsplattform; du lieferst nur deine Anwendung.	Google App Engine, SAP BTP
SaaS – Software as a Service	fertige Anwendung über den Browser, im Abo; nichts selbst betreiben.	**Salesforce** (CRM, SaaS-Pionier – Folie), Microsoft 365

- **Ursprung:** Amazon musste hohe Rechenleistung für Weihnachts- und „Back2School"-Geschäft vorhalten und fragte sich, wie man sie sonst nutzt bzw. monetarisiert → **AWS seit 2002**.
- **Hyperscaler (Auswahl):** Amazon (AWS), Microsoft (Azure), Google Cloud (GCP), STACKIT (Schwarz-Gruppe, Neckarsulm, seit 2022).
- **Eigenschaften:** flexible virtuelle Instanzen verschiedener Betriebssysteme, skalierbar; Hadoop verteilt parallelisierbare Last. Beispiel: Die New York Times wandelte mit gemieteten AWS-Instanzen ca. 11 Mio. Artikel in 24 Stunden in PDF um.

Treiber des technischen Fortschritts (nur Folie) (teilweise Rückschau)

Treiber	Aussage
Mooresches Gesetz	siehe Block 3
Digitale Massenspeicher	exponentieller Kapazitätszuwachs bei drastisch fallenden Preisen; Datenmenge verdoppelt sich grob jedes Jahr (Lyman/Varian 2003)
Gesetz von Metcalfe	bei n Teilnehmern $n\cdot(n-1)$ mögliche Verbindungen → Wert eines Netzes wächst überproportional mit der Teilnehmerzahl. Beispiel: 10 Teilnehmer → $10\cdot9 = 90$ (gerichtete) Verbindungen. *(Ergänzung: ungerichtet $n\cdot(n-1)/2 = 45$, vgl. „alle stoßen an" in Block 8.)*
Sinkende Kommunikationskosten	nähern sich 0; über 5 Mrd. Menschen online; keine freien IPv4-Adressen mehr
Standardisierung	Spezifikationen für Kompatibilität → Skaleneffekte, Preissenkungen (Ethernet, TCP/IP, HTTP, ASCII, HTML …)

⚠ TYPISCHE KLAUSURFALLEN

- Es sind **sieben** Komponenten – die oft vergessenen sind „Internet als Plattform" und „Beratungsdienste/Systemintegratoren".
- Epochen **lösen sich nicht ab**, sie überlagern sich (Großrechner gibt es heute noch).
- Client = Eingabestelle des Benutzers, Server = gemeinsame Daten/Dienste – nicht umgekehrt.
- Cloud: Daten liegen **nicht** mehr lokal im Unternehmen; Bezug über das Internet, meist im Abo.

DAS MUSST DU KÖNNEN

- ☐ 7 Komponenten der IT-Infrastruktur nennen
- ☐ 5 Epochen in Reihenfolge mit je einem Merkmal
- ☐ Client-Server und N-Tier erklären/skizzieren
- ☐ Cloud Computing definieren, IaaS/PaaS/SaaS unterscheiden
- ☐ Hyperscaler nennen, Ursprung AWS
- ☐ Metcalfe $n\cdot(n-1)$ anwenden

3 · Mooresches Gesetz — 2 P

IN EINEM SATZ Moore: Die Zahl der Transistoren pro Chip verdoppelt sich regelmäßig, und Verdoppeln heißt jedes Mal „mal 2“, nicht „plus etwas“.

STELL DIR VOR Du **faltest ein Blatt Papier**: 1× gefaltet 2 Lagen, 2× 4, 3× 8, nach 10 Faltungen $2^{10} = 1024$ Lagen. So schnell wächst alles, was sich regelmäßig verdoppelt.

Was Moore tatsächlich gesagt hat (Rückschau)

Gordon Moore, damals Direktor der Forschungslabore von Fairchild Semiconductor und später Mitgründer von **Intel**, schrieb **1965** im *Electronics Magazine*, dass sich seit dem ersten Mikrochip (1959) die **Anzahl der (niedrigpreisigen) Komponenten, im Allgemeinen Transistoren, auf einem Chip jährlich verdoppelt** habe, und sagte das für die nächsten 10 Jahre voraus. Später **korrigierte er die Rate auf eine Verdopplung alle zwei Jahre**. Diese Aussage begründet das Mooresche Gesetz. Es beschreibt eine **exponentielle** Zunahme der Verarbeitungsleistung bei gleichzeitig sinkenden Kosten.

Interpretationen – keine davon stammt von Moore selbst (Rückschau)

1. Die Leistung von **Mikroprozessoren** verdoppelt sich alle **18 Monate**.
2. Die Leistung von **Computern** verdoppelt sich alle **18 Monate**.
3. Der **Preis** für rechnergestützte Informationsverarbeitung **halbiert** sich alle **18 Monate**.

Begründung: **Fläche** ist der wichtigste Kostenfaktor bei Mikrochips – mehr Transistoren auf gleicher Fläche bedeuten mehr Leistung pro Euro. **Grenzen** (VL01): Kosten pro Transistor steigen bei sehr kleinen Strukturen wieder, Energie- und Kühlungsbedarf wächst. **Auswege**: neue Materialien und Lithographie (ASML, TSMC), 3D-Chipdesign, Quantencomputer und vor allem **Skalierung über mehrere GPUs/Cluster/Rechenzentren** (Stand 2025: keine „größeren“ Chips mehr). Beispiel Nvidia Blackwell: 208 Mrd. Transistoren (TSMC 3 nm).

Rechnen mit exponentiellem Wachstum *(Ergänzung zur Veranschaulichung)*

Verdopplung alle T Jahre → nach t Jahren Faktor $2^{t/T}$. Beispiele: 10 Jahre bei Verdopplung alle 2 Jahre → 2^5 = **32-fach**. 6 Jahre bei Verdopplung alle 18 Monate → 2^4 = **16-fach**.

Beispiel-MC aus VL05 (2 Punkte) mit Lösung

Antwortoption: „Korrekte Interpretation des Mooreschen Gesetzes?“	Lösung	Warum
Die Anzahl der Transistoren auf einem Chip **verdreifacht** sich alle 1–2 Jahre	×	verdoppelt, nicht verdreifacht
Die **Leistung von Mikroprozessoren verdoppelt sich alle ca. 18 Monate**	✓	Interpretation 1 der Folie
Die in einem Computer verbaute **Speicherkapazität** verdoppelt sich regelmäßig alle 18 Monate	×	keine der drei Interpretationen
Die **Größe** eines Transistors halbiert sich alle 1–2 Jahre	×	keine der Interpretationen (Fläche ≠ Anzahl)

Punktelogik: Wer nur Option 2 ankreuzt, bekommt 4 × ½ = 2 Punkte. Wer zusätzlich Option 3 ankreuzt: 3 × ½ − ½ = 1 Punkt.

⚠ TYPISCHE KLAUSURFALLEN

- **Original** (Moore): **Transistoren pro Chip** verdoppeln sich – erst jährlich, später alle **2 Jahre**. **Interpretationen** (nicht Moore): Leistung bzw. Kosten, **18 Monate**. Zahlen und Bezugsgrößen nicht mischen.
- Die Fallen stecken in einzelnen Wörtern: verdreifacht statt verdoppelt, Speicherkapazität statt Leistung, Größe statt Anzahl.
- „Kosten halbieren sich“ ist richtig (Interpretation 3), „Kosten verdoppeln sich“ falsch.
- Moore war bei Fairchild, später **Intel**-Gründer (nicht AMD, nicht IBM).

DAS MUSST DU KÖNNEN

- ☐ Moores ursprüngliche Aussage mit Jahreszahl 1965 wiedergeben
- ☐ Korrektur auf 2 Jahre kennen
- ☐ alle 3 Interpretationen nennen
- ☐ „Fläche ist wichtigster Kostenfaktor“ erklären
- ☐ Grenzen und Auswege nennen
- ☐ MC-Optionen wortgenau prüfen

4 · Zahlendarstellung und Rechnen 3 P

IN EINEM SATZ Wie viel eine Ziffer wert ist, hängt von ihrer Stelle ab; negative Zahlen entstehen im Zweierkomplement durch Rückwärtszählen ab 0.

STELL DIR VOR Du bezahlst 235 € mit 100er-, 10er- und 1er-Scheinen; im Binärsystem gibt es nur die Scheine 1, 2, 4, 8, 16 …, von jedem höchstens einen. **Zweierkomplement = rückwärts laufender Kilometerzähler**: Ein dreistelliger Zähler auf 000, einen Kilometer zurück → 999 (= −1), noch einen → 998 (= −2). Mit 8 Bit: 0000 0000 − 1 = 1111 1111 = −1. Fährt der Zähler über 999 hinaus, springt er auf 000: Das ist der **Überlauf**.

Der Dozent: „**DAS IST ÜBUNGSSACHE!!**“ und „**sauber schreiben**“. Kernaufgaben sind (1) Umwandlung zwischen den Basen 2, 8, 10, 16, (2) binäres Rechnen mit positiven und negativen ganzen Zahlen, (3) Darstellung nicht ganzzahliger Werte.

Stellenwertsysteme Rückschau

In einem Stellenwertsystem hängt der Wert einer Ziffer von ihrer **Position** ab. Jede Stelle hat die Wertigkeit $\text{Basis}^{\text{Position}}$, von rechts beginnend bei Position 0. Beispiel: $235_{10} = 2{\cdot}10^2 + 3{\cdot}10^1 + 5{\cdot}10^0$. (Das römische System hat keine Stellenwerte und keine Null.)

System	Basis	Ziffern	235_{10} =
Dual/Binär	2	0, 1	1110 1011
Oktal	8	0–7	353
Dezimal	10	0–9	235
Hexadezimal	16	0–9, A–F	EB

dez	bin	okt	hex	dez	bin	okt	hex
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

Zweierpotenzen auswendig: 2^0=1, 2^1=2, 2^2=4, 2^3=8, 2^4=16, 2^5=32, 2^6=64, 2^7=128, 2^8=256, 2^9=512, 2^{10}=1024. Sechzehnerpotenzen: 16^0=1, 16^1=16, 16^2=256, 16^3=4096. Achterpotenzen: 1, 8, 64, 512.

Verfahren A: beliebige Basis → dezimal (Stellenwerte aufsummieren)

```
1110 1011₂ = 1·128 + 1·64 + 1·32 + 0·16 + 1·8 + 0·4 + 1·2 + 1·1 = 235
353₈       = 3·64 + 5·8 + 3·1   = 192 + 40 + 3   = 235
EB₁₆       = 14·16 + 11·1       = 224 + 11       = 235
2AF₁₆      = 2·256 + 10·16 + 15 = 512 + 160 + 15 = 687
753₈       = 7·64 + 5·8 + 3     = 448 + 40 + 3   = 491
```

Verfahren B: dezimal → Basis b (Restwertmethode) Rückschau

Zahl fortlaufend **ganzzahlig durch die Basis teilen** und jeweils den **Rest** notieren, bis der Quotient 0 ist. Die Reste **von unten nach oben** gelesen ergeben die Zahl. Tipp der Folie: die Reste gleich von rechts nach links notieren.

```
235 : 2 = 117  Rest 1   ← letzte Stelle
117 : 2 =  58  Rest 1
 58 : 2 =  29  Rest 0
 29 : 2 =  14  Rest 1
 14 : 2 =   7  Rest 0
  7 : 2 =   3  Rest 1
  3 : 2 =   1  Rest 1
  1 : 2 =   0  Rest 1   ← erste Stelle
von unten lesen: 1110 1011₂
```

```
235 : 8 = 29  Rest 3
 29 : 8 =  3  Rest 5
  3 : 8 =  0  Rest 3
→ 353₈

235 : 16 = 14  Rest 11 (= B)
 14 : 16 =  0  Rest 14 (= E)
→ EB₁₆
```

Verfahren C: binär ↔ oktal/hex durch Gruppenbildung Rückschau

Weil 8 = 2^3 und 16 = 2^4 ist, entspricht eine Oktalziffer genau **3 Bit**, eine Hexziffer genau **4 Bit**. Gruppen immer **von rechts** bilden und links mit Nullen auffüllen. Oktal ↔ Hex geht am einfachsten über den Umweg binär.

```
1110 1011₂  → Dreiergruppen: 011 | 101 | 011  →  3 5 3₈
            → Vierergruppen: 1110 | 1011      →  E B₁₆
1335₈ → binär: 001 011 011 101 = 1011011101₂ → Vierergruppen 0010 | 1101 | 1101 → 2DD₁₆ (= 733₁₀)
```

Übungsaufgaben von den Folien (gelöst)

- 458_{10} = **$1\,1100\,1010_2$** = **712_8** (Dreiergruppen 111 001 010).
- $1001\ 0010_2$ = **222_8** (010 010 010) = **92_{16}** (1001 0010) = 146_{10}.
- Was ist größer: $E7E5_{16}$, 59364_{10} oder $1110\ 0111\ 1110\ 0110_2$? → $E7E5_{16}$ = 59365, die Binärzahl = $E7E6_{16}$ = 59366. **Die Binärzahl ist am größten**, 59364 am kleinsten. (Tipp: Binär direkt in Hex gruppieren und nur die letzte Ziffer vergleichen.)

▶ REZEPT: DEZIMAL ↔ BINÄR ↔ HEX/OKTAL (KOMPLETT DURCHGERECHNET: 173)

1. **Dezimal → binär:** Restwertmethode, durch 2 teilen, Reste von **unten nach oben** lesen.
2. **Binär → hex/oktal:** von rechts in **4er-** (hex) bzw. **3er-Gruppen** (oktal) teilen, links mit 0 auffüllen, jede Gruppe einzeln übersetzen.
3. **Hex/oktal → binär:** jede Ziffer als 4 bzw. 3 Bit hinschreiben ($3F_{16} \rightarrow 0011\ 1111_2$).
4. **Irgendwas → dezimal:** Ziffer × Stellenwert, alles addieren.
5. **Probe:** Ergebnis auf dem anderen Weg zurückrechnen.

Kurzweg Dezimal → Hex: direkt durch 16 teilen (173 : 16 = 10 Rest 13 → D; 10 : 16 = 0 Rest 10 → A → **AD_{16}**).

```
① 173 : 2 = 86 R 1
  86 : 2 = 43 R 0
  43 : 2 = 21 R 1
  21 : 2 = 10 R 1
  10 : 2 =  5 R 0
   5 : 2 =  2 R 1
   2 : 2 =  1 R 0
   1 : 2 =  0 R 1
↑ von unten lesen:
1010 1101₂
```

```
② 1010 1101₂
  4er: 1010 | 1101
       A  |  D   → AD₁₆
  3er: 010 | 101 | 101
       2  |  5  |  5 → 255₈

④ Probe:
  AD₁₆ = 10·16 + 13 = 173 ✓
  255₈ = 2·64 + 5·8 + 5 = 173 ✓
  128+32+8+4+1 = 173 ✓
```

Binäre Addition Rückschau

Regeln: 0+0 = 0, 0+1 = 1, 1+0 = 1, **1+1 = 0 Übertrag 1** (und 1+1+1 = 1 Übertrag 1). Wie schriftliches Addieren von rechts nach links.

```
  0100 0001   (65)
+ 0101 1010   (90)
Übertrag  1
= 1001 1011   (155)
```

```
    111 1111   (127)
+   111 1111   (127)
= 1111 1110   (254)  ← Folienaufgabe:
  Ergebnis braucht ein Bit mehr!
```

Der **Datentyp** legt die Bitbreite und damit den Wertebereich fest. Passt das Ergebnis nicht mehr hinein, entsteht ein **Überlauf**.

Negative Zahlen – drei Darstellungen (8 Bit, Beispiel ±42) Rückschau

Darstellung	Bildung von −42	+42 / −42	Wertebereich (8 Bit)	Eigenschaften
Vorzeichen und Betrag	erstes Bit = Vorzeichen (0 +, 1 −), 7 Bit Betrag	0 0101010 1 0101010	−127 … +127	sehr anschaulich; **zwei Nullen** (+0, −0); Addition braucht Sonderregeln/Fallunterscheidungen
Einerkomplement	alle Bits invertieren	0 0101010 1 1010101	−127 … +127	einfach; **zwei Nullen**; Übertrag aus dem höchsten Bit muss unten wieder addiert werden (**End-around Carry**)
Zweierkomplement (Standard)	alle Bits invertieren, **dann +1**	0 0101010 1 1010110	**−128 … +127**	**nur eine Null**; höchstes Bit hat Wertigkeit **−128**; Addition und Subtraktion mit **derselben** Addierschaltung, letzter Übertrag wird ignoriert

Allgemein für n Bit *(Ergänzung)*: Zweierkomplement -2^{n-1} … $2^{n-1}-1$ (8 Bit: −128 … 127; 16 Bit: −32768 … 32767 = Java `short`). Vorzeichen-Betrag und Einerkomplement: $-(2^{n-1}-1)$ … $2^{n-1}-1$.

Zweierkomplement Schritt für Schritt

1. **Bitbreite festlegen** (z. B. 8 Bit). Ohne feste Bitbreite ist das Zweierkomplement nicht definiert.
2. Positive Zahl binär mit führenden Nullen auf volle Breite: 17 = `0001 0001`.
3. Alle Bits invertieren (Einerkomplement): `1110 1110`.
4. **+1** addieren: `1110 1111` = −17.
5. **Probe** über die Wertigkeiten: −128 + 64 + 32 + 8 + 4 + 2 + 1 = −17 ✓. Rückweg: negative Zahl erneut invertieren und +1 → Betrag.

Wert	42	−17	−42	0	−1	127	−128
8 Bit	`0010 1010`	`1110 1111`	`1101 0110`	`0000 0000`	`1111 1111`	`0111 1111`	`1000 0000`

▶ REZEPT: BITMUSTER IM ZWEIERKOMPLEMENT → DEZIMALZAHL

1. **Erstes Bit ansehen.** 0 → positiv, ganz normal umrechnen, fertig.
2. 1 → negativ. **Weg A:** höchstes Bit zählt **−128** (bei 8 Bit), die übrigen Bits normal addieren.
3. **Weg B:** invertieren, +1, Betrag ablesen, Minus davor.
4. Beide Wege müssen dasselbe liefern (Probe).

```
1011 0100₂ (8 Bit), erstes Bit 1 → negativ
Weg A: −128 + 32 + 16 + 4       = −76
Weg B: invertieren  0100 1011  (= 75)
       +1           0100 1100  (= 76) → −76 ✓
Merke: 1000 0000 = −128, 1111 1111 = −1
```

Subtraktion = Addition des Zweierkomplements (Folienaufgaben, 8 Bit)

```
42 − 17 = 42 + (−17)
   0010 1010   (42)
 + 1110 1111   (−17)
 (1)0001 1001  → Übertrag (1) weg
 = 0001 1001 = 25 ✓
```

```
−42 − 17:
   1101 0110   (−42)
 + 1110 1111   (−17)
 = (1)1100 0101 = −59 ✓
   Probe: −128+64+4+1 = −59
```

```
42 − 42:                                  −1 − 17:
   0010 1010 + 1101 0110                     1111 1111 + 1110 1111
 = (1)0000 0000 = 0 ✓                      = (1)1110 1110 = −18 ✓
```

Folienbeispiel mit **5 Bit**: 14 + (−9): 9 = 01001 → invertiert 10110 → +1 = **10111**; 01110 + 10111 = (1)00101 = **5** ✓.

Überlauf erkennen

Der weggeworfene **Übertrag** aus dem höchsten Bit ist **kein** Fehler. Ein **Überlauf** liegt vor, wenn das echte Ergebnis außerhalb des Wertebereichs liegt. Faustregel *(Ergänzung)*: Überlauf genau dann, wenn **beide Summanden dasselbe Vorzeichen** haben, das **Ergebnis aber das andere**. Zwei Zahlen mit verschiedenen Vorzeichen können nie überlaufen.

```
100 + 50 (8 Bit):          −100 + (−50):              127 + 1:
   0110 0100   (+100)         1001 1100   (−100)        0111 1111
 + 0011 0010   (+50)        + 1100 1110   (−50)      + 0000 0001
 = 1001 0110   → −106 ✗     = (1)0110 1010 → +106 ✗  = 1000 0000 → −128 ✗
 positiv + positiv = negativ negativ + negativ = positiv ÜBERLAUF
 → ÜBERLAUF (150 > 127)     → ÜBERLAUF (−150 < −128)  (typischer MC-Fall)
```

Einerkomplement mit End-around Carry (Folie, 4 Bit): −4 + 6 = 1011 + 0110 = (1)0001 → Übertrag 1 unten addieren: 0001 + 1 = **0010 = +2** ✓. Bei −4 + 3 = 1011 + 0011 = 1110 (−1) entsteht kein Übertrag.

▶ REZEPT: BINÄR ADDIEREN UND ÜBERLAUF PRÜFEN

1. Beide Zahlen auf **dieselbe Bitbreite** bringen; negative Zahlen vorher ins Zweierkomplement.
2. Untereinander schreiben, **von rechts** Spalte für Spalte addieren; eine **Übertragszeile** darüber führen (1+1 = 0 Ü 1, 1+1+1 = 1 Ü 1).
3. Übertrag aus dem höchsten Bit **streichen** (Zweierkomplement).
4. **Überlauf-Test**: Haben beide Summanden dasselbe Vorzeichenbit und das Ergebnis ein anderes? → Überlauf. Verschiedene Vorzeichen → nie Überlauf.
5. Ergebnis zurückrechnen und mit der Dezimalrechnung vergleichen.

Das Beispiel rechts zeigt: **Dasselbe Bitmuster** ist ohne Vorzeichen richtig (145 ≤ 255), mit Vorzeichen ein Überlauf (145 > 127). Ob ein Überlauf vorliegt, hängt also vom **Datentyp** ab.

```
91 + 54 in 8 Bit
Übertrag   1111 11··
            0101 1011   (+91)
          + 0011 0110   (+54)
          = 1001 0001

ohne Vorzeichen: 128+16+1 = 145 ✓
Zweierkomplement: 0 + 0 → 1
 → −128+16+1 = −111 ✗ ÜBERLAUF

−20 + (−30) in 8 Bit:
   1110 1100 + 1110 0010
 = (1)1100 1110 → Ü streichen
 1 + 1 → 1: kein Überlauf
 −128+64+8+4+2 = −50 ✓
```

Exkurs: Shift-Operator (nur Folie)

Verschieben um 1 Bit nach links = ×2, nach rechts = ganzzahlig ÷2 (für vorzeichenlose Ganzzahlen). Java: 23 = 0001 0111; `23 << 1` = 0010 1110 = 46; `23 << 2` = 0101 1100 = 92; `23 >> 1` = 11. Rechenwerke haben dafür Shift-Register.

Nicht ganzzahlige Werte (nur Folie) (Exkurs in VL02, aber Kernaufgabe in VL05)

Binäre Kommazahlen (Festkomma)

Nach dem Komma haben die Stellen die Wertigkeiten 2^{-1} = ½, 2^{-2} = ¼, 2^{-3} = ⅛ usw. Vorkommateil wie gewohnt umrechnen; den **Nachkommateil fortlaufend mit 2 multiplizieren**, die Vorkommastelle (0 oder 1) ist das nächste Bit, weiter mit dem Rest, bis 0 herauskommt. Die Bits **von oben nach unten** lesen.

```
4,375₁₀:  4 = 100₂
0,375 · 2 = 0,75 → 0   (2⁻¹)
0,75  · 2 = 1,5  → 1   (2⁻²)
0,5   · 2 = 1,0  → 1   (2⁻³)  fertig
0,375 = 0·½ + 1·¼ + 1·⅛ = 0,011₂
4,375₁₀ = 100,011₂
```

```
0,4₁₀:
0,4·2=0,8→0  0,8·2=1,6→1
0,6·2=1,2→1  0,2·2=0,4→0
0,4·2=0,8→0  … wiederholt sich
0,4₁₀ = 0,0110 0110 0110…₂ (periodisch!)
→ nicht exakt darstellbar
```

Mit **fester** Anzahl Vor- und Nachkommabits nennt man das **Festkommadarstellung** *(Begriff = Ergänzung)*: einfach, aber kleiner Wertebereich.

Gleitkomma (Fließkomma, Floating Point)

Eine Gleitkommazahl besteht aus drei Teilen: **Mantisse m** (üblicherweise nicht ganzzahliger Anteil), **Basis b** und **Exponent e**: $\mathbf{Z = m \cdot b^e}$. Beispiel: 2022 = 2,022 · 10^3. Vorteil: Der darstellbare Zahlenraum wird um nicht ganzzahlige Zahlen **und** durch die Potenzen in seiner Größe erweitert. Gerechnet wird mit Coprozessoren oder speziellen Prozessorbefehlen.

IEEE 754	Vorzeichen	Exponent	Mantisse	Bias	Java
einfache Genauigkeit, 32 Bit	1 Bit	8 Bit	23 Bit	127	`float`
doppelte Genauigkeit, 64 Bit	1 Bit	11 Bit	52 Bit	1023	`double`

```
Folienbeispiel 18,4 in 32 Bit:
1. Vorzeichen:       0 (positiv)
2. Vorkomma:         18 = 10010₂ ;  Nachkomma 0,4 = 0,0110 0110 …₂
```

```
3. Zwischenergebnis: 10010,0110 0110 0110 … · 2⁰
4. Normalisieren:    1,0010 0110 0110 … · 2⁴   → Exponent 4 (Komma 4 Stellen nach links)
5. Mantisse:         0010 0110 0110 … (die führende 1 = „Hidden Bit“ wird nicht gespeichert)
6. Charakteristik:   Exponent + Bias = 4 + 127 = 131 = 1000 0011₂
7. Ergebnis:         0 | 10000011 | 00100110011001100110011
   gespeichert ist eigentlich 18,3999996…  → Ungenauigkeit durch „Abschneiden“ der Mantisse!
```

⚠ TYPISCHE KLAUSURFALLEN

- Restwertmethode: Reste **von unten nach oben** lesen. Bei Hex die Reste 10–15 als **A–F** schreiben.
- Gruppen für Oktal (3) und Hex (4) immer **von rechts** bilden, links mit Nullen auffüllen.
- Zweierkomplement immer mit der **angegebenen Bitbreite** (führende Nullen!). −17 in 8 Bit ist `1110 1111`, nicht `101111`.
- Zweierkomplement = invertieren **plus 1**; Einerkomplement = nur invertieren. Positive Zahlen bleiben unverändert.
- Wertebereich 8 Bit Zweierkomplement: **−128 bis +127** (asymmetrisch!), Vorzeichen-Betrag/Einerkomplement: −127 bis +127 mit zwei Nullen.
- **Übertrag ≠ Überlauf.** Der Übertrag aus dem höchsten Bit wird im Zweierkomplement ignoriert; Überlauf erkennst du am Vorzeichen.
- 0,4 oder 0,1 sind binär **periodisch** → Gleitkommazahlen sind oft ungenau.
- IEEE 754: Exponent wird **mit Bias** gespeichert (4 → 131), die führende 1 der Mantisse wird weggelassen.

DAS MUSST DU KÖNNEN

- ☐ 2 ↔ 8 ↔ 10 ↔ 16 in alle Richtungen umrechnen
- ☐ Restwertmethode sauber aufschreiben
- ☐ binär addieren mit Überträgen
- ☐ ±x in Vorzeichen-Betrag, Einer- und Zweierkomplement
- ☐ a − b im Zweierkomplement rechnen
- ☐ Überlauf erkennen und begründen
- ☐ Wertebereiche nennen
- ☐ Dezimalbruch → binäre Kommazahl
- ☐ Gleitkomma $m \cdot b^e$ und IEEE-Aufbau erklären

5 · Wahrheitstabellen, Boolesche Ausdrücke, Logikgatter

4 P

IN EINEM SATZ Boolesche Algebra ist Rechnen mit „ja“ (1) und „nein“ (0); jede Regel aus UND, ODER, NICHT lässt sich als Tabelle, Ausdruck und Schaltung schreiben.

STELL DIR VOR **AND** = Bankschließfach mit zwei Schlüsseln (beide nötig). **OR** = Haustür mit Klingel vorne und hinten (eine reicht). **NOT** = Schalter, der das Gegenteil macht. **De Morgan**: „Nicht (Kaffee und Kuchen)“ heißt „kein Kaffee oder kein Kuchen“.

„**AUCH ÜBUNGSSACHE!!**“ Aufgabentypen laut VL05: Wahrheitstabelle aus textueller Beschreibung aufstellen; Booleschen Ausdruck aus Wertetabelle aufstellen; einfache Schaltung aus **genormten Logikbausteinen** aus Tabelle oder Ausdruck entwickeln (Analyse und Synthese).

Boolesche Algebra und Grundoperatoren Rückschau

Die Boolesche Algebra (George Boole, 19. Jh.) kennt nur die Werte **0 (falsch)** und **1 (wahr)**. Sie ist die Grundlage der Digitaltechnik: In Chips übernehmen **Transistoren** die Rolle von Schaltern. AND entspricht einer Reihenschaltung, OR einer Parallelschaltung.

Operator	Schreibweisen	Bedeutung
AND (Konjunktion, logisches Produkt)	A ∧ B, A · B, A & B	1 nur, wenn **alle** Eingänge 1 sind
OR (Disjunktion, logische Summe)	A ∨ B, A + B	1, wenn **mindestens ein** Eingang 1 ist
NOT (Negation, Invertierung)	¬A, Ā	kehrt den Wert um

Die sieben Gatter – Wahrheitstabelle und Symbole nach IEC 60617 Rückschau

A	B	AND	OR	XOR	NAND	NOR	XNOR	NOT A
0	0	0	0	0	1	1	1	1
0	1	0	1	1	1	0	0	1
1	0	0	1	1	1	0	0	0
1	1	1	1	0	0	0	1	0
Merkhilfe		alle 1	mind. eine 1	„ungleich“	nicht AND	nicht OR	„gleich“	umkehren

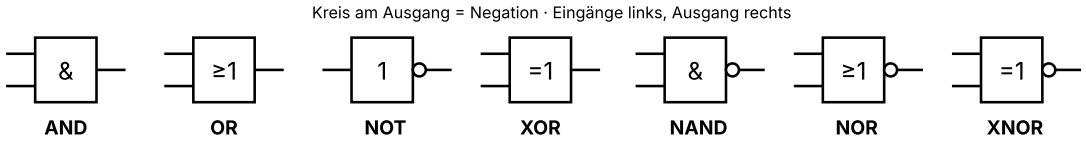


Abb. 5.1 – Logikgatter nach IEC 60617 (wie auf der Folie markiert). In Europa gilt IEC 60617, in US-Schaltplänen oft ANSI 91-1984 (runde/spitze Formen); DIN 40700 ist veraltet.

De-Morgan-Gesetze und NAND als Universalgatter

> **De Morgan:** ¬(A ∧ B) = ¬A ∨ ¬B und ¬(A ∨ B) = ¬A ∧ ¬B. Merksatz: **Negation über die Klammer ziehen → AND und OR tauschen, jede Variable negieren.**

A	B	A ∧ B	¬(A ∧ B)	¬A	¬B	¬A ∨ ¬B
0	0	0	**1**	1	1	**1**
0	1	0	**1**	1	0	**1**
1	0	0	**1**	0	1	**1**
1	1	1	**0**	0	0	**0**

NAND (ebenso NOR) ist vollständig: Allein damit lässt sich jede logische Funktion bauen – praktisch für die Chipfertigung und FPGAs. Folienübung „AND nur aus NAND“:

- **NOT A** = NAND(A, A) (beide Eingänge zusammenschalten).
- **A AND B** = NOT(NAND(A, B)) = NAND(NAND(A,B), NAND(A,B)) → 2 NAND-Gatter.
- **A OR B** = NAND(¬A, ¬B) = NAND(NAND(A,A), NAND(B,B)) → 3 NAND-Gatter (Begründung De Morgan: ¬(¬A ∧ ¬B) = A ∨ B).

Synthese: Wahrheitstabelle → Boolescher Ausdruck (DNF) → Schaltung Rückschau

Das „Standard-Vorgehen“ der Folie heißt in der Fachsprache **disjunktive Normalform (DNF)** *(Begriff = Ergänzung)*:

1. Alle Zeilen suchen, in denen der **Ausgang 1** ist.

2. Für jede dieser Zeilen einen **UND-Term** bilden: Variable mit Wert 1 → **A**, Variable mit Wert 0 → **¬A**. Der Term ist genau in dieser Zeile 1.
3. Alle UND-Terme mit **ODER** verknüpfen.
4. Optional vereinfachen (z. B. Spezialgatter wie XNOR erkennen oder ausklammern).
5. Schaltung: Inverter für negierte Eingänge → je Term ein &-Gatter → ein ≥1-Gatter am Ende.

Folienbeispiele: Tabelle mit Out = 1 nur bei A=1, B=0 → **A ∧ ¬B**. Tabelle mit Out = 1 nur bei A=0, B=1 → **B ∧ ¬A**. Tabelle mit Out = 1 bei (0,0) und (1,1):

A	B	Out	Teilterm
0	0	**1**	¬A ∧ ¬B
0	1	0	
1	0	0	
1	1	**1**	A ∧ B

Ergebnis: **Out = (¬A ∧ ¬B) ∨ (A ∧ B)**
Abkürzung: Das ist genau **XNOR** („gleich“) → ein einziges Gatter =1 mit Negationskreis.

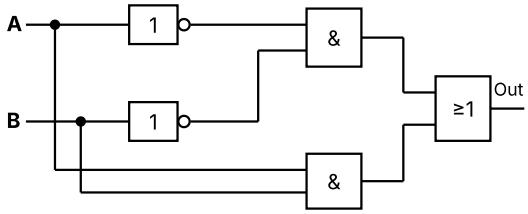


Abb. 5.2 – Standard-Vorgehen als Schaltung (Folie 24)

Von der textuellen Beschreibung zur Schaltung – durchgerechnetes Beispiel *(eigenes Übungsbeispiel)*

„Eine Alarmanlage (Y) soll auslösen, wenn sie scharf geschaltet ist (S) und die Tür (T) oder das Fenster (F) geöffnet ist.“

S	T	F	Y	UND-Term
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	**1**	S ∧ ¬T ∧ F
1	1	0	**1**	S ∧ T ∧ ¬F
1	1	1	**1**	S ∧ T ∧ F

1. Tabelle: 3 Eingänge → 2^3 = **8 Zeilen**; Eingänge wie Binärzahlen 000 bis 111 hochzählen.

2. DNF: Y = (S ∧ ¬T ∧ F) ∨ (S ∧ T ∧ ¬F) ∨ (S ∧ T ∧ F)

3. Vereinfachen: S steckt in jedem Term → ausklammern; die drei Rest-Terme sind genau dann 1, wenn T oder F gilt: **Y = S ∧ (T ∨ F)**. Beide Ausdrücke liefern dieselbe Tabelle (mit Python geprüft). Die DNF ist immer richtig, wenn die Zeit knapp ist.

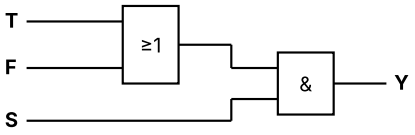


Abb. 5.3 – Y = S ∧ (T ∨ F)

Boolesche Ausdrücke vereinfachen – Rechengesetze *(Gesetzesnamen = Ergänzung)*

Gesetz	UND-Form	ODER-Form
Kommutativ	a ∧ b = b ∧ a	a ∨ b = b ∨ a
Distributiv (ausklammern)	a ∧ (b ∨ c) = (a ∧ b) ∨ (a ∧ c)	a ∨ (b ∧ c) = (a ∨ b) ∧ (a ∨ c)
Idempotenz	a ∧ a = a	a ∨ a = a
Neutrales Element	a ∧ 1 = a	a ∨ 0 = a
Extremal	a ∧ 0 = 0	a ∨ 1 = 1
Komplement	a ∧ ¬a = 0	a ∨ ¬a = 1
Doppelte Negation	¬¬a = a	
De Morgan	¬(a ∧ b) = ¬a ∨ ¬b	¬(a ∨ b) = ¬a ∧ ¬b
Absorption	a ∧ (a ∨ b) = a	a ∨ (a ∧ b) = a

Kernregel (daraus abgeleitet): Zwei UND-Terme, die sich nur in **einer** Variablen unterscheiden (einmal x, einmal ¬x), verschmelzen, und x fällt weg:

```
(A ∧ B) ∨ (A ∧ ¬B)
= A ∧ (B ∨ ¬B)   Distributiv
= A ∧ 1          Komplement
= A              Neutral
```

Stell dir vor: „Ich gehe raus, wenn die Sonne scheint und es warm ist, und auch, wenn die Sonne scheint und es nicht warm ist.“ Dann ist die Temperatur egal.

▶ REZEPT: AUSDRUCK MIT GESETZEN VEREINFACHEN

1. **Negationen über Klammern** zuerst mit De Morgan auflösen, dann ¬¬x = x.
2. **Paare suchen**: Welche UND-Terme unterscheiden sich in genau einer Variablen? Gemeinsames ausklammern, x ∨ ¬x = 1 streichen.
3. Ein Term darf **mehrfach** benutzt werden (x ∨ x = x).
4. Absorption und Extremalgesetze anwenden, bis nichts mehr geht.
5. **Probe**: eine 1-Zeile und eine 0-Zeile einsetzen (oder ganze Tabelle).

Muster erkennen: (¬A ∧ B) ∨ (A ∧ ¬B) = XOR, (¬A ∧ ¬B) ∨ (A ∧ B) = XNOR – nicht kürzbar, aber je ein Gatter.

```
Bsp. 1  (¬A∧B) ∨ (A∧B) ∨ (A∧¬B)     A∧B doppelt nutzen
  (¬A∧B) ∨ (A∧B) = B ∧ (¬A∨A) = B
  (A∧B) ∨ (A∧¬B) = A ∧ (B∨¬B) = A
  → A ∨ B   (= OR, Probe A=0,B=0 → 0 ✓)

Bsp. 2  Alarmanlage (siehe oben)
  (S∧¬T∧F) ∨ (S∧T∧F) = S∧F
  (S∧T∧¬F) ∨ (S∧T∧F) = S∧T
  → (S∧F) ∨ (S∧T) = S ∧ (T ∨ F)

Bsp. 3  ¬(A ∧ ¬B) ∨ B
  = ¬A ∨ ¬¬B ∨ B   De Morgan
  = ¬A ∨ B ∨ B     Doppelnegation
  = ¬A ∨ B         Idempotenz

Bsp. 4  ¬(A ∨ ¬B) ∨ (A ∧ B)
  = (¬A ∧ B) ∨ (A ∧ B)  De Morgan, ¬¬B = B
  = B ∧ (¬A ∨ A) = B    Kernregel
```

Analyse: Ausdruck oder Schaltung → Wahrheitstabelle

Umgekehrt gehst du so vor: Spalten für alle Eingänge (2^n Zeilen), dann **Hilfsspalten für Zwischenergebnisse** (z. B. Ausgang jedes Gatters), zuletzt der Ausgang. Beispiel Y = (A ∨ B) ∧ ¬C: Hilfsspalten A ∨ B und ¬C; Y ist 1 in den Zeilen (A,B,C) = (0,1,0), (1,0,0), (1,1,0), sonst 0.

▶ REZEPT: LOGIKGATTER-SCHALTUNG ANALYSIEREN (SCHALTUNG → AUSDRUCK → TABELLE)

1. Jeden **Gatterausgang benennen** (X1, X2, …), von den Eingängen nach rechts.
2. Für jedes Gatter den **Teilausdruck** hinschreiben (Kreis am Ausgang = Negation), am Ende zum Gesamtausdruck einsetzen.
3. Tabelle mit 2^n Zeilen (000 … 111), **je Gatter eine Hilfsspalte**, Zeile für Zeile von links nach rechts füllen.
4. Ausgangsspalte ablesen; auf Wunsch DNF aus den 1-Zeilen bilden und vereinfachen.

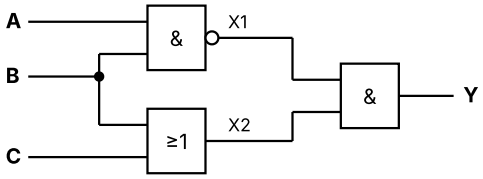


Abb. 5.4 – Übungsschaltung: NAND, OR, AND *(eigenes Beispiel)*

```
X1 = ¬(A ∧ B)    (NAND: & mit Kreis)
X2 = B ∨ C
Y  = X1 ∧ X2 = ¬(A ∧ B) ∧ (B ∨ C)
```

A	B	C	X1	X2	Y
0	0	0	1	0	0
0	0	1	1	1	**1**
0	1	0	1	1	**1**
0	1	1	1	1	**1**
1	0	0	1	0	0
1	0	1	1	1	**1**
1	1	0	0	1	0
1	1	1	0	1	0

DNF der 1-Zeilen: (¬A∧¬B∧C) ∨ (¬A∧B∧¬C) ∨ (¬A∧B∧C) ∨ (A∧¬B∧C). Kernregel: Zeilen 001 + 101 → **¬B ∧ C**, Zeilen 010 + 011 → **¬A ∧ B**. Also **Y = (¬B ∧ C) ∨ (¬A ∧ B)** (mit Python gegen die Schaltung geprüft).

Rechnen mit Gattern: Halbaddierer und Volladdierer (Rückschau)

A	B	s	c_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Ein **Halbaddierer** addiert zwei Bits: Summe **s = A XOR B**, Übertrag **c = A AND B**. Ein **Volladdierer** berücksichtigt zusätzlich einen eingehenden Übertrag (c_{in}) und besteht aus **zwei Halbaddierern und einem OR-Gatter**. Mehrere Volladdierer hintereinander, die den Übertrag weiterreichen, ergeben ein **Addierwerk** (z. B. 4 Bit).

Folienhinweis (Exkurs Flipflop, VL02): Ein Flipflop (bistabile Kippstufe aus zwei rückgekoppelten NOR- oder NAND-Gattern) speichert 1 Bit: Set → 1, Reset → 0, S = R = 0 → Zustand bleibt. Die Zustandstabelle auf der Folie vertauscht S und R – im Zweifel nach dieser Regel gehen. Der Flipflop-Exkurs steht nicht in der Rückschau.

⚠ TYPISCHE KLAUSURFALLEN

- Bei n Eingängen hat die Tabelle **2^n Zeilen** (3 Eingänge = 8 Zeilen). Systematisch wie Binärzahlen hochzählen, sonst vergisst du Zeilen.
- In der DNF wird eine **0 in der Zeile zu ¬Variable**, eine 1 zur Variable. Nur Zeilen mit Ausgang 1 verwenden.
- XOR ist 0 bei (1,1) – „entweder oder". XNOR = „gleich".
- De Morgan: ¬(A ∧ B) ist **nicht** ¬A ∧ ¬B, sondern ¬A **∨** ¬B.
- IEC-Symbole: AND = **&**, OR = **≥1**, XOR = **=1**, NOT = **1** mit Kreis. Negation = **Kreis am Ausgang**. Nicht die US-Formen mischen.
- Halbaddierer: Summe = XOR (nicht OR!), Übertrag = AND.

DAS MUSST DU KÖNNEN

- ☐ Wahrheitstabellen aller 7 Gatter aufschreiben
- ☐ IEC-Symbole zeichnen
- ☐ Text → Wahrheitstabelle
- ☐ Tabelle → DNF-Ausdruck
- ☐ Ausdruck → Schaltung mit &, ≥1, 1
- ☐ Schaltung/Ausdruck → Tabelle (Hilfsspalten)
- ☐ De Morgan anwenden
- ☐ AND/OR/NOT nur aus NAND bauen
- ☐ Halbaddierer erklären

6 · Programmierung 2 P

IN EINEM SATZ Ein Algorithmus ist ein Rezept mit endlich vielen Schritten, ein Programm das aufgeschriebene Rezept in einer Sprache, ein Prozess das Rezept, das gerade gekocht wird.

STELL DIR VOR Ein **Compiler** übersetzt wie ein Übersetzer ein ganzes Buch vorab; ein **Interpreter** ist ein Dolmetscher, der Satz für Satz live übersetzt. **Rekursion**: In einer Schlange fragst du deinen Vordermann nach seinem Platz, der fragt seinen … bis die Erste sagt „Platz 1“ (Abbruchbedingung); auf dem Rückweg zählt jeder 1 dazu.

Laut VL01: „Dies ist **KEIN** Programmierkurs.“ Die Klausur stellt hier „allgemeine Fragen“ – Begriffe, Unterschiede, Rekursion.

Algorithmus, Programm, Prozess (Rückschau)

Begriff	Definition (Folie)	Beispiel
Algorithmus	Handlungsvorschrift zur Lösung eines Problems oder Erreichen eines Ziels **in endlicher Zeit**. Grafisch modellierbar als **Struktogramm** (erlaubt kein GOTO) oder **UML-Aktivitätsdiagramm**.	Kuchenrezept; Zahlen sortieren (Folie: Selection Sort als Struktogramm)
Programm	Umgesetzte Version eines oder mehrerer Algorithmen in einer bestimmten Programmiersprache; darf auch **unendlich** laufen.	Serverprozess; „Gib jede Sekunde eine um 1 erhöhte Zahl aus“ = Programm, aber **kein** Algorithmus (nicht endlich)
Prozess	Ein **laufendes** Programm mit allen Daten, Variablenwerten und Verwaltungsinformationen des Betriebssystems (→ 2. Semester).	geöffneter Browser im Taskmanager

Von der Maschinensprache zur Hochsprache (Rückschau)

Ein Computer kann nur **binäre Maschinenbefehle** ausführen. Diese sind für Menschen praktisch unlesbar und **je nach Prozessor verschieden**. Die erste Abstraktion war **Assembler**: symbolische Namen für Maschinenbefehle (z. B. `ADD`, beim 6502 `LDA NUM1 / ADC NUM2 / STA RES`: Wert ins Register laden, addieren, Ergebnis an eine Adresse schreiben). Mit wachsender Komplexität wurde Assembler unübersichtlich; **höhere Programmiersprachen** (Python, Java, C++) formulieren Algorithmen in für Menschen verständlicher Form und brauchen ein Übersetzungsprogramm. Entwicklungslinie (nur Folie): BASIC (GOTO) → strukturierte Programmierung (Pascal) → Objektorientierung/Kapselung (C++, Java) → Speichersicherheit (Rust). Der **TIOBE-Index** misst die **Popularität** einer Sprache, nicht ihre Qualität.

Werkzeuge der Softwareentwicklung (nur Folie) (im Rückblick VL03a genannt)

Werkzeug	Aufgabe
Compiler	übersetzt das Programm **vorab** vollständig in Maschinencode für eine bestimmte Hardware. Phasen: Scanner (lexikalische Analyse) → Parser (Syntaxanalyse, Syntaxbaum) → semantische Analyse/Optimierung → Codeerzeugung.
Interpreter	übersetzt und führt die Befehle erst **zur Laufzeit** aus; Optimierung durch Vorübersetzen in maschinenunabhängigen **Bytecode** (Java, Python).
Linker	verbindet getrennt übersetzte Module zu einem ausführbaren Programm. **Statisch**: alles in eine Datei; **dynamisch**: Bibliotheken (DLL) erst zur Laufzeit – teilbar und austauschbar, aber Abbruch, wenn eine fehlt.
Lader	lädt den Maschinencode an eine Speicheradresse und passt die Adressen an; beim Rechnerstart: Bootstrap.
Debugger / IDE	schrittweise Ausführung mit Haltepunkten / Editor + Compiler + Debugger in einer Umgebung (z. B. Eclipse).

Fehlerarten: Syntax, Semantik, Typen (nur Folie)

- **Syntaxfehler**: falsche Schreibweise (z. B. fehlende Klammer) → der Compiler meldet einen Fehler.
- **Semantikfehler**: korrekt geschrieben, aber falsche Bedeutung (z. B. Werte vertauscht) → **kein** Fehler gemeldet, Ergebnis falsch.
- **Typfalle in Java**: `int z1 = 10; int z2 = 3; double res = z1 / z2;` ergibt **3.0**, weil erst `int / int` ganzzahlig geteilt (3) und dann in double umgewandelt wird. Richtig: `(double) z1 / z2` → 3.333…

> **Folienfehler (VL02 F. 86):** `double(z1) / z2` ist kein gültiges Java (C++-Schreibweise). In Java heißt der Cast `(double) z1`; außerdem steht dort `Int` statt `int`.

Elementare Datentypen (Java) (nur Folie)

Typ	byte	short	int	long	float	double	char	boolean
Größe	8 Bit	16 Bit	32 Bit	64 Bit	32 Bit	64 Bit	16 Bit (Unicode)	1 Bit (laut Folie)
Bereich	−128…127	−32768…32767	ca. ±2,1 Mrd.	ca. $\pm 9{,}2 \cdot 10^{18}$	Gleitkomma IEEE 754		Zeichen	true/false

Rekursion: Teile und Herrsche Rückschau

Teile und Herrsche (Divide and Conquer): Eine Aufgabe wird in (zwei) gleichartige Teilaufgaben mit verkleinerter (halbierter) Eingabe zerlegt (z. B. Merge Sort). **Rekursion** ist der Implementierungsaspekt: Eine Funktion ruft sich (ggf. mehrfach) **selbst** mit geänderten Eingabewerten auf. Sie braucht eine **Abbruchbedingung**, sonst ruft sie sich endlos auf (Stack-Überlauf).

```
// iterativ (Schleife)
public static long fakite(int number) {
  long sum = 1;
  for (int i = 1; i <= number; i++) {
    sum = sum * i;
  }
  return sum;
}
```

```
// rekursiv (Selbstaufruf)
public static long fakrek(int number) {
  if (number == 1 || number == 0) {
    return 1;          // Abbruchbedingung
  }
  else return number * fakrek(number - 1);
}
```

fakrek(4) = 4 · fakrek(3) = 4 · 3 · fakrek(2) = 4 · 3 · 2 · fakrek(1) = 4 · 3 · 2 · 1 = 24
N! = 1 · 2 · … · N; 0! = 1! = 1. Türme von Hanoi: n Scheiben brauchen $2^n - 1$ Züge (3 Scheiben → 7).

⚠ TYPISCHE KLAUSURFALLEN

- Ein Algorithmus muss **endlich** sein, ein Programm nicht (Endlosschleife/Server = Programm, kein Algorithmus).
- Programm ≠ Prozess: Prozess = **laufendes** Programm mit Daten und Verwaltungsinformationen.
- Maschinensprache ist **prozessorabhängig**; Assembler ist nur eine symbolische Schreibweise davon.
- Compiler übersetzt vorab, Interpreter zur Laufzeit. Syntaxfehler meldet der Compiler, Semantikfehler nicht.
- Rekursion ohne Abbruchbedingung terminiert nicht.

DAS MUSST DU KÖNNEN

- ☐ Algorithmus/Programm/Prozess abgrenzen
- ☐ Maschinensprache/Assembler/Hochsprache
- ☐ Compiler vs. Interpreter
- ☐ Syntax- vs. Semantikfehler
- ☐ Rekursion + Abbruchbedingung erklären
- ☐ Fakultät rekursiv nachvollziehen

7 · Datenstrukturen

6–8 P

IN EINEM SATZ Datenstrukturen sind feste Ordnungen, nach denen ein Programm Daten im Speicher ablegt und wieder herausholt.

STELL DIR VOR **Stack = Tellerstapel** (der zuletzt aufgelegte Teller kommt zuerst weg, LIFO). **Queue = Warteschlange an der Kasse** (wer zuerst kommt, ist zuerst dran, FIFO). **Array** = Wand mit nummerierten Schließfächern, **verkettete Liste** = Schnitzeljagd (jeder Zettel verrät nur den nächsten), **Binärbaum** = Zahlenraten „kleiner oder größer?“.

Gefragt werden laut VL05 „**Bedeutung, Funktionsweise und APIs**“ sowie die „Anwendung der Datenstrukturen Stack, Queue, Liste, sortierte Liste, binäre Bäume“. Du musst also Operationen ausführen können („Was liegt nach push(3), push(7), pop() oben?“) und Vor-/Nachteile begründen.

Elementare Datentypen vs. Datenstrukturen Rückschau

Elementare Datentypen sind die Grundbausteine, z. B. eine ganze Zahl mit 32 Bit oder ein Zeichen mit 1 Byte. **Datenstrukturen** dienen der strukturierten Ablage von Daten im Hauptspeicher: Mehrere Daten (auch verschiedener Typen) werden gemeinsam adressiert, Schachtelungen sind möglich, und die Struktur hängt vom Anwendungskontext ab. Manche sind in der Sprache vorgegeben, andere frei definierbar.

- **String**: Folge von Zeichen (char) im Speicher; Operationen: vergleichen, Zeichen anfügen, Teile auslesen.
- **Selbstdefinierte Struktur „Kunde“**: ID-Nummer (Int), Name (String), durchschnittlicher Umsatz/Jahr (Float). So eine „flache“ Struktur entspricht einer Zeile in einer relationalen Datenbank.
- **Tabelle** (Vorgriff auf SQL): Kombination aus selbstdefinierter Struktur und Array/Liste; eine Spalte enthält einen **eindeutigen Schlüssel** (Kundennummer).

Array (Feld) Rückschau

Älteste Datenstruktur, vergleichbar mit Vektoren/Matrizen. Elemente werden unter einem **fortlaufenden Index** gespeichert (beginnend bei 0) und können direkt darüber angesprochen werden. Arrays haben eine **feste Länge**, die beim Anlegen festgelegt wird; mehrdimensional möglich.

Index	0	1	2	3	…	9
Wert	Walter	Heinz	Frank	Karl	…	Horst

Vorteile: einfach; effizienter **direkter Zugriff** über den Index. **Nachteile:** Länge nicht direkt änderbar (in neues Array umkopieren); Einfügen und Entfernen nicht direkt möglich (Elemente umkopieren/verschieben).

Verkettete Listen Rückschau

In einer **einfach verketteten Liste** gibt es nicht zwingend einen Index. Bekannt ist nur die Position des **ersten Elements** (Kopf); alle weiteren Elemente können verstreut im Speicher liegen, und **jedes Element kennt nur seinen Nachfolger** (Zeiger). Neue Elemente werden dynamisch angelegt und durch Umhängen von Zeigern eingefügt. Die **Suche** erfordert das Durchlaufen ab dem Startelement.

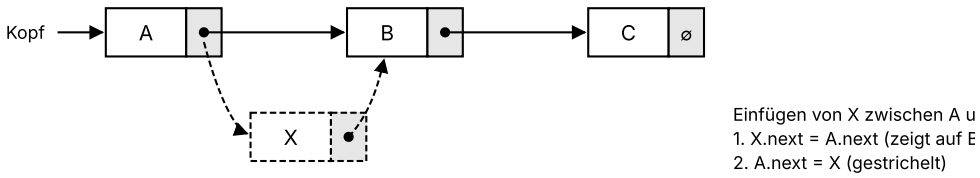


Abb. 7.1 – Einfach verkettete Liste; Einfügen = nur zwei Zeiger umhängen, nichts umkopieren. Löschen von B: A.next = B.next.

Bei der **doppelt verketteten Liste** kennt jedes Element **Vorgänger und Nachfolger**. Bestimmte Operationen werden einfacher: Kennt man das zu löschende Element schon, muss man nicht erst die Liste durchlaufen, um den Vorgänger zu finden.

Sortierte Liste *(steht in der VL05-Themenliste, Erklärung = Ergänzung)*: Die Elemente werden beim Einfügen gleich an der **richtigen Position** (nach dem Sortierkriterium) eingehängt. Dafür muss man die Liste ab dem Kopf durchlaufen, bis das erste größere Element kommt (Aufwand O(n)); dafür ist die Liste jederzeit sortiert, und die Suche kann abbrechen, sobald ein größeres Element erreicht ist. Beispiel: In 10 → 19 → 36 wird 25 zwischen 19 und 36 eingehängt. Binäre Suche lohnt sich auf einer verketteten Liste nicht, weil man die Mitte nicht direkt per Index anspringen kann, sondern hinlaufen muss (vgl. Folienbild „19?“).

Stack (Stapel) und Queue (Warteschlange) Rückschau

Stack: spezielle Liste ohne freien Zugriff auf die Elemente; Prinzip **LIFO** (Last In, First Out) – wie ein Tellerstapel.

Queue: listenartige Struktur, Prinzip **FIFO** (First In, First Out) – wie eine Warteschlange an der Kasse.

- `put(x)`: Element **am Ende** einfügen (oft auch `enqueue`)

- `push(x)`: neues Element oben auflegen
- `pop()`: oberstes Element entnehmen
- `top()`: oberstes Element lesen, aber liegen lassen
- Fehlerfall: `pop()`/`top()` auf leerem Stapel

Anwendungen *(Ergänzung)*: Funktionsaufrufe/Rekursion (Aufrufstack), „Rückgängig“, Klammerprüfung.

- `get()`: Element **am Anfang** entnehmen (oft auch `dequeue`)
- Fehlerfall: `get()` auf leerer Queue

Anwendungen *(Ergänzung)*: Druckerwarteschlange, Nachrichten zwischen Systemen, Aufträge der Reihe nach abarbeiten.

Operation	Stack (oben = rechts)	Ausgabe	Operation	Queue (vorne = links)	Ausgabe
`push(3)`	`[3]`		`put(3)`	`[3]`	
`push(7)`	`[3, 7]`		`put(7)`	`[3, 7]`	
`pop()`	`[3]`	**7**	`get()`	`[7]`	**3**
`push(1)`	`[3, 1]`		`put(1)`	`[7, 1]`	
`top()`	`[3, 1]`	**1** (bleibt)	`put(9)`	`[7, 1, 9]`	
`push(9)`	`[3, 1, 9]`		`get()`	`[1, 9]`	**7**
`pop(); pop()`	`[3]`	**9, 1**	`get()`	`[9]`	**1**

▶ REZEPT: STACK- UND QUEUE-TRACES SICHER DURCHSPIELEN

1. Tabelle mit drei Spalten anlegen: **Operation | Inhalt danach | Ausgabe**. Vorher festlegen und dazuschreiben, wo „oben“ (Stack) bzw. „vorne“ (Queue) ist.
2. **Stack:** push legt rechts (oben) an, pop nimmt rechts weg und **gibt es aus**, top gibt rechts aus, **ohne** zu entfernen.
3. **Queue:** put/enqueue hängt rechts (hinten) an, get/dequeue nimmt links (vorne) weg und gibt es aus.
4. Jede Zeile einzeln ausführen, nie zwei Operationen im Kopf zusammenfassen. Bei pop/get auf leerer Struktur „Fehler (leer)“ notieren.
5. Am Ende die **Ausgabefolge** und den **Restinhalt** getrennt angeben.

Stack (oben = rechts)	Inhalt	Ausgabe
`push(4)`	`[4]`	
`push(8)`	`[4, 8]`	
`pop()`	`[4]`	**8**
`push(5)`	`[4, 5]`	
`push(2)`	`[4, 5, 2]`	
`pop()`	`[4, 5]`	**2**
`top()`	`[4, 5]`	**5** (bleibt)
`push(9)`	`[4, 5, 9]`	
`pop(); pop()`	`[4]`	**9, 5**
Ausgabe 8, 2, 5, 9, 5 · Rest [4]		

Queue (vorne = links)	Inhalt	Ausgabe
`put(4)`	`[4]`	
`put(8)`	`[4, 8]`	
`get()`	`[8]`	**4**
`put(5)`	`[8, 5]`	
`put(2)`	`[8, 5, 2]`	
`get()`	`[5, 2]`	**8**
`put(9)`	`[5, 2, 9]`	
`get(); get()`	`[9]`	**5, 2**
Ausgabe 4, 8, 5, 2 · Rest [9]		

Typische Kombi-Aufgabe: 1, 2, 3, 4 per put in eine Queue, dann alle per get herausnehmen und sofort push auf einen Stack, danach alles pop: Queue liefert 1, 2, 3, 4 (Reihenfolge bleibt), der Stack gibt **4, 3, 2, 1** aus. **Ein Stack kehrt die Reihenfolge um, eine Queue nicht.**

Klammerprüfung mit Stack *(Ergänzung)*: öffnende Klammer → push; schließende → pop und vergleichen. „([ ] { })“: am Ende Stack leer → korrekt. „([ )]“: bei „)“ liefert pop „[“ → passt nicht → **fehlerhaft**.

Bäume – Begriffe (Rückschau)

Bäume sind eine **zweidimensionale Erweiterung verketteter Listen**. Ein Baum ist eine nicht-leere Menge von **Knoten** und **Kanten** (Verbindungen zwischen zwei Knoten). Die **Wurzel** ist der Ursprung; ein **Pfad** ist eine Folge unterschiedlicher, durch Kanten verbundener Knoten.

- Jeder Knoten außer der Wurzel hat **genau einen Vorgänger** und 0 bis n Nachfolger.
- **Blätter** (Endknoten): keine Nachfolger. **Innere Knoten**: mindestens ein Nachfolger.
- Jeder Knoten ist Wurzel eines **Unterbaums**.
- **Ebene** eines Knotens = Anzahl Kanten von der Wurzel bis zu ihm; **Höhe** = höchste Ebene; **Pfadlänge** = Summe der Ebenen aller Knoten.
- **Wald** = Menge von Bäumen; **n-ärer Baum** = höchstens n Nachfolger.
- **Binärbaum**: höchstens zwei Nachfolger (links/rechts).
- **Geordnet** (binärer Suchbaum): Jeder Knoten ist **größer als sein linker** und **kleiner als sein rechter** Nachfolger – also links kleiner, rechts größer.
- **Voll** (Folie, (nur Folie)): keine „Halbblätter“ (Knoten mit genau einem Nachfolger).
- **Vollständig** (Folie): voller Baum, bei dem alle Blätter auf der letzten Ebene liegen.
- **Balanciert**: Blätter nur auf letzter/vorletzter Ebene, Tiefe ≈ $\log_2(n+1)$. **Degeneriert**: jeder Knoten hat höchstens ein Kind → praktisch eine Liste.

Wozu? Binäre Bäume vereinen die Vorteile von Arrays (**schnelles Auffinden**) mit denen von Listen (**leichtes Einfügen/Entfernen**). Operationen: **Suchen, Einfügen, Löschen**.

Binärbaum aufbauen, suchen, löschen – Folienaufgabe gelöst

Einfügen: An der Wurzel beginnen; ist der neue Wert kleiner, nach links, sonst nach rechts gehen, bis ein freier Platz kommt; dort als Blatt anhängen. **Suchen** genauso: pro Ebene ein Vergleich. **Löschen** – drei Fälle: (1) Blatt →

einfach entfernen; (2) ein Kind → das Kind rückt an die Stelle; (3) zwei Kinder → durch den **größten Knoten im linken** oder den **kleinsten Knoten im rechten Teilbaum** ersetzen (Folienregel).

Aufgabe: Binärbaum aus **[26, 5, 18, 16, 1, 43, 39, 27, 12]** (in dieser Reihenfolge einfügen):

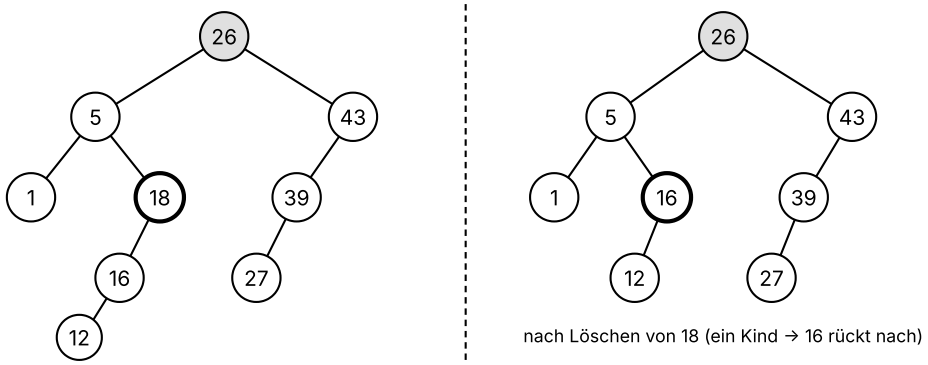


Abb. 7.2 – Folienaufgabe Bäume. Wurzel grau, der Knoten mit dickem Rand ist der gelöschte bzw. nachgerückte.

- **Aufbau**: 26 Wurzel; 5 < 26 → links; 18: <26, >5 → rechts von 5; 16: <26, >5, <18 → links von 18; 1 → links von 5; 43 → rechts von 26; 39 → links von 43; 27 → links von 39; 12: <26, >5, <18, <16 → links von 16.
- **18 löschen** (hat nur ein Kind, 16): 16 rückt mit seinem Unterbaum nach. Der Baum ist danach **weder voll** (Halbblätter 16, 43, 39) **noch vollständig**.
- **26 löschen** (zwei Kinder): Variante „größter links" → **16** wird Wurzel: 16(5(1, 12), 43(39(27))). Variante „kleinster rechts" → **27** wird Wurzel: 27(5(1, 16(12)), 43(39)).
- **12 löschen** (Blatt): Variante 16 → 16(5(1), 43(39(27))); Variante 27 → 27(5(1, 16), 43(39)). In beiden Fällen **weder voll noch vollständig** (es gibt Halbblätter).
- **Degeneriert**: Fügt man bereits sortierte Werte ein (1, 2, 3, 4), hängt jeder Knoten rechts am vorherigen → Liste, Suche O(n) statt O(log n).

Folienbeispiel „Aufbau": Einfügen 7, 13, 11, 5, 14, 2 ergibt 7(5(2), 13(11, 14)). *(Ergänzung: Liest man links – Knoten – rechts („Inorder"), erhält man die sortierte Folge 2, 5, 7, 11, 13, 14.)*

Vergleich der Datenstrukturen

Struktur	Prinzip	Operationen (API)	Zugriff/Suche	Einfügen/Löschen	Vorteil / Nachteil
Array	feste Länge, Index 0…n−1	a[i] lesen/schreiben	direkt über Index (O(1)); Suche linear oder binär (wenn sortiert)	nur durch Umkopieren	einfach, schneller Indexzugriff / starre Länge
Einfach verkettete Liste	Element + Zeiger auf Nachfolger; nur Kopf bekannt	einfügen, löschen, durchlaufen	nur sequentiell ab Kopf (O(n))	Zeiger umhängen, kein Umkopieren	dynamische Größe / langsame Suche
Doppelt verkettete Liste	Zeiger auf Vorgänger und Nachfolger	wie oben, in beide Richtungen	sequentiell (O(n))	Löschen ohne Vorgängersuche	bequemer / mehr Speicher für Zeiger
Sortierte Liste	Elemente stets geordnet	sortiert einfügen, löschen, suchen	sequentiell, Abbruch beim ersten größeren	Einfügeposition suchen (O(n))	immer sortiert / Einfügen teurer
Stack	**LIFO**	push, pop, top	nur oberstes Element	nur oben	sehr einfach / kein freier Zugriff
Queue	**FIFO**	put (enqueue), get (dequeue)	nur vorderstes Element	hinten rein, vorne raus	faire Reihenfolge / kein freier Zugriff
Geordneter Binärbaum	links kleiner, rechts größer	suchen, einfügen, löschen	pro Ebene 1 Vergleich, balanciert O(log n)	leicht (Blatt anhängen / Ersatzknoten)	schnell suchen + leicht einfügen / kann degenerieren

Die O(…)-Angaben in dieser Tabelle sind teils *Ergänzung*; auf den Folien stehen O(n) und O(log n) bei den Suchverfahren (Block 8).

⚠ TYPISCHE KLAUSURFALLEN

- **Stack = LIFO, Queue = FIFO** – nicht vertauschen. `top()` entfernt **nicht**, `pop()` schon.
- Auf den Folien heißen die Queue-Operationen **Put()/Get()**, in VL05 werden auch enqueue/dequeue genannt – beide Namen kennen.
- Immer an den **Fehlerfall** denken: pop/top/get auf leerer Struktur.
- Array: **feste** Länge, Index beginnt bei **0**. Liste: kein Index, nur der Kopf ist bekannt.
- Binärbaum: Gleiche Zahlenfolge in anderer Reihenfolge → anderer Baum. Reihenfolge genau einhalten.
- Löschen mit zwei Kindern: Ersatz ist der **größte im linken** oder **kleinste im rechten** Teilbaum – nicht einfach das linke Kind.
- „Voll" und „vollständig" nach Foliendefinition beantworten (voll = keine Halbblätter).

DAS MUSST DU KÖNNEN

- ☐ Array vs. Liste mit Vor-/Nachteilen
- ☐ Einfügen/Löschen in Liste erklären (Zeiger)
- ☐ einfach vs. doppelt verkettet
- ☐ sortierte Liste erklären
- ☐ Stack: push/pop/top, LIFO, Fehlerfall
- ☐ Queue: put/get, FIFO, Fehlerfall
- ☐ Operationsfolgen „durchspielen“
- ☐ Baumbegriffe (Wurzel, Blatt, Ebene, Höhe)
- ☐ Binärbaum aus Zahlenfolge aufbauen
- ☐ Knoten löschen (3 Fälle)
- ☐ voll / vollständig / degeneriert beurteilen

8 · Suchen und Sortieren

6 P

IN EINEM SATZ Suchen und Sortieren werden schnell, wenn man bei jedem Schritt einen großen Teil der Daten ausschließen kann; die O-Notation misst, wie die Arbeit mit der Datenmenge wächst.

STELL DIR VOR **Binäre Suche = Telefonbuch** in der Mitte aufschlagen und eine Hälfte wegwerfen. **Bubble Sort** = Lehrerin tauscht in der Reihe immer zwei Nachbarn, bis das größte Kind ganz rechts steht. **Merge Sort** = zwei sortierte Klausurstapel zusammenlegen, immer das kleinere oberste Blatt zuerst. **O(n²)** = jeder Gast verabschiedet sich von jedem.

Gefragt werden die „**Eigenschaften der verschiedenen Verfahren**“: Voraussetzungen, Funktionsweise, Aufwand (O-Notation). Zu den meistgenutzten Algorithmen zählen laut Folie: Sortieren (Mergesort, Quicksort, naiv Bubblesort), Suchen (lineare und binäre Suche), Graphen/kürzeste Wege, Klassifikation/Machine Learning/Data Mining.

O-Notation – wie skaliert ein Algorithmus? (nur Folie) (Exkurs, aber bei allen Verfahren verwendet)

Die O-Notation beschreibt die **asymptotische Laufzeit** in Abhängigkeit von der Eingabegröße n: Wie wächst der Aufwand, wenn die Datenmenge wächst? Konstante Faktoren werden weggelassen.

Klasse	Name	Beispiele (Folie)	n = 1.000	n = 1.000.000
O(1)	konstant	Addition von 10 Zahlen	1	1
O(log n)	logarithmisch	binäre Suche in sortiertem Array	≈ 10	≈ 20
O(n)	linear	lineare Suche in unsortierter Liste; Skalarprodukt	1.000	10^6
O(n log n)	„n log n“	Merge Sort	≈ 10.000	≈ $2 \cdot 10^7$
O(n²)	quadratisch	Bubble Sort („jeder mit jedem“); alle stoßen an: $n \cdot (n-1)/2$	10^6	10^{12}

Folienfehler (VL02 F. 138): Dort steht die Multiplikation zweier Matrizen unter $O(n^2)$. Für n×n-Matrizen ist das naive Verfahren **$O(n^3)$** – so steht es auch korrekt im KI-Exkurs VL03c.

Suchverfahren (Rückschau)

Lineare Suche

Es werden **alle** Elemente des Arrays oder der Liste nacheinander angesprungen und mit dem Suchkriterium verglichen. Geeignet auch für **unsortierte** Daten. Der Aufwand steigt **linear** mit der Anzahl der Elemente: **O(n)**.

Binäre Suche

Setzt voraus, dass die Daten bereits **sortiert** sind. Man prüft das **mittlere** Element; ist der Suchwert größer, sucht man in der rechten Hälfte weiter, ist er kleiner, in der linken. Die Datenmenge wird so wiederholt halbiert; der Aufwand ist auf die Anzahl der Halbierungen begrenzt: **O(log n)**.

```
Array (sortiert): Index  0   1   2   3   4   5   6   7   8
                  Wert  10  19  23  25  36  61  79  81  99
Suche 61:  Mitte Index 4 = 36 → 61 > 36 → rechts weiter (Index 5–8)
           Mitte Index 6 = 79 → 61 < 79 → links weiter (Index 5)
           Index 5 = 61 → gefunden nach 3 Vergleichen  (Folie identisch)
Suche 19:  Mitte 36 → 19 < 36 → links (Index 0–3); Mitte (0+3)/2 = 1 → 19 gefunden (2 Vergleiche)
Suche 80:  36 → 79 → 81 → Bereich leer → nicht vorhanden (3 Vergleiche statt 9 bei linearer Suche)
```

Bei gerader Anzahl gibt es zwei „Mitten“ – welche man nimmt, ist Konvention. Die Folie wählt bei 19 im linken Teil 23 und braucht 3 Schritte; das Ergebnis ist dasselbe. Merke: 1.000 Elemente → max. ca. 10 Vergleiche, 1 Mio. → ca. 20.

Sortierverfahren

Bubble Sort (Rückschau)

Je zwei **benachbarte** Elemente werden verglichen und, falls sie in der falschen Reihenfolge stehen, **vertauscht**. Am Ende des ersten Durchlaufs steht das **größte Element am Ende** („steigt auf wie eine Blase“). Es sind (höchstens) so viele Durchläufe nötig, wie es Elemente gibt. Der Aufwand steigt **quadratisch**: **O(n²)**.

```
Start:        [5, 1, 4, 2, 8]
Durchlauf 1:  5↔1 → [1,5,4,2,8]; 5↔4 → [1,4,5,2,8]; 5↔2 → [1,4,2,5,8]; 5<8 bleibt  → [1, 4, 2, 5, 8]  (8 sitzt fest)
Durchlauf 2:  1<4; 4↔2 → [1,2,4,5,8]; 4<5                                  → [1, 2, 4, 5, 8]
Durchlauf 3:  keine Vertauschung mehr → fertig (Abbruch möglich, Ergänzung)
```

Merge Sort (Rückschau)

Das Array wird in **zwei Hälften** geteilt, beide Hälften werden **jede für sich (rekursiv) sortiert**, anschließend werden die beiden bereits sortierten Hälften wieder **zusammengeführt („Merge“)**: Man vergleicht jeweils die vordersten Elemente beider Hälften und übernimmt das kleinere. Prinzip **Teile und Herrsche**, implementiert über **Rekursion**. Aufwand **O(n log n)**.

```
[38, 27, 43, 3, 9, 82, 10]
Teilen:   [38, 27, 43]           |  [3, 9, 82, 10]
          [38] [27, 43]          |  [3, 9]   [82, 10]
               [27] [43]         |  [3] [9]  [82] [10]
Mergen:   [27, 43] → [27, 38, 43] |  [3, 9]   [10, 82] → [3, 9, 10, 82]
Merge:    [27, 38, 43] + [3, 9, 10, 82] → [3, 9, 10, 27, 38, 43, 82]
```

▶ REZEPT: SORTIER-DURCHLÄUFE AUFSCHREIBEN (BEISPIEL [29, 10, 14, 37, 13])

1. **Bubble Sort:** Pro Durchlauf von links je zwei **Nachbarn** vergleichen, bei falscher Reihenfolge tauschen. Nach jedem Durchlauf ist das größte unsortierte Element hinten fest (Strich ziehen). Durchlauf ohne Tausch → fertig.
2. **Merge Sort:** halbieren bis Einzelelemente, dann paarweise mergen: immer die vordersten Elemente vergleichen, das kleinere übernehmen.
3. **Selection Sort** *(Erg.)*: im unsortierten Rest das **Minimum** suchen und mit dem ersten Element des Rests tauschen.
4. Pro Durchlauf **eine Zeile** mit dem kompletten Array hinschreiben, Tauschpaare markieren.

Zählprobe Bubble Sort: bei n = 5 höchstens 4 + 3 + 2 + 1 = **10 Vergleiche** = n·(n−1)/2 → O(n²).

```
Bubble Sort                              Vergl. Tausch
D1: 29↔10 · 29↔14 · 29<37 · 37↔13
    [10, 14, 29, 13 | 37]                 4     3
D2: 10<14 · 14<29 · 29↔13
    [10, 14, 13 | 29, 37]                 3     1
D3: 10<14 · 14↔13
    [10, 13 | 14, 29, 37]                 2     1
D4: 10<13, kein Tausch → fertig           1     0
    [10, 13, 14, 29, 37]

Selection Sort (Minimum nach vorne)
S1: Min 10 ↔ 29  → [10 | 29, 14, 37, 13]
S2: Min 13 ↔ 29  → [10, 13 | 14, 37, 29]
S3: Min 14 bleibt → [10, 13, 14 | 37, 29]
S4: Min 29 ↔ 37  → [10, 13, 14, 29, 37]

Merge Sort
[29, 10] [14, 37, 13] → [29][10] [14][37, 13]
→ [10, 29]  [14] [13, 37] → [13, 14, 37]
→ Merge [10, 29] + [13, 14, 37]
  10 · 13 · 14 · 29 · 37 → [10, 13, 14, 29, 37]
```

Weitere Verfahren *(auf den Folien nur genannt – Erklärung = Ergänzung)*

- **Quicksort** (in der Liste der meistgenutzten Algorithmen): wählt ein Pivot-Element, teilt in „kleiner als Pivot“ und „größer als Pivot“ und sortiert beide Teile rekursiv; im Mittel O(n log n), im schlechtesten Fall O(n²).
- **Selection Sort** (Folienbeispiel für Struktogramm/UML-Diagramm): sucht jeweils das kleinste Element im unsortierten Rest und stellt es nach vorne; O(n²).

Vergleich der Such- und Sortierverfahren

Verfahren	Typ	Voraussetzung	Prinzip	Aufwand	stabil? *(Ergänzung)*
Lineare Suche	Suchen	keine (auch unsortiert)	alle Elemente nacheinander prüfen	**O(n)**	–
Binäre Suche	Suchen	**sortiert**, direkter Zugriff (Array)	Mitte prüfen, Hälfte verwerfen	**O(log n)**	–
Suche im Binärbaum	Suchen	geordneter Binärbaum	links kleiner, rechts größer	O(log n) balanciert, O(n) degeneriert	–
Bubble Sort	Sortieren	–	Nachbarn vergleichen und tauschen	**O(n²)**	ja
Merge Sort	Sortieren	–	Teile und Herrsche, rekursiv, Merge	**O(n log n)**	ja (braucht Zusatzspeicher)
Quicksort *(Erg.)*	Sortieren	–	Pivot, Teile und Herrsche	Ø O(n log n), schlechtester Fall O(n²)	nein
Selection Sort *(Erg.)*	Sortieren	–	Minimum suchen, nach vorne tauschen	O(n²)	nein

Stabil *(Ergänzung)* heißt: Elemente mit gleichem Sortierschlüssel behalten ihre ursprüngliche Reihenfolge (wichtig, wenn man z. B. erst nach Name und dann nach Abteilung sortiert).

⚠ TYPISCHE KLAUSURFALLEN

- Binäre Suche funktioniert **nur auf sortierten** Daten. Die lineare Suche geht immer.
- Binäre Suche = O(**log n**), nicht O(n log n). Merge Sort = O(**n log n**), Bubble Sort = O(**n²**).
- Bubble Sort: Nach dem 1. Durchlauf steht das **größte** Element am **Ende** (nicht das kleinste vorne).
- Merge Sort teilt zuerst und sortiert dann – das eigentliche Sortieren passiert beim **Zusammenführen**.
- „Teile und Herrsche“ ist das Prinzip, „Rekursion“ die Implementierung – nicht gleichsetzen.
- Bei Wachstumsfragen: Verdopplung von n verdoppelt O(n), vervierfacht O(n²), erhöht O(log n) nur um 1 Schritt.

DAS MUSST DU KÖNNEN

- ☐ lineare vs. binäre Suche: Voraussetzung, Aufwand
- ☐ binäre Suche Schritt für Schritt
- ☐ Bubble Sort einen Durchlauf zeigen
- ☐ Merge Sort Teilen/Mergen skizzieren
- ☐ Teile und Herrsche / Rekursion erklären
- ☐ O(1), O(log n), O(n), O(n log n), $O(n^2)$ zuordnen
- ☐ meistgenutzte Algorithmen nennen

9 · Rechnerarchitektur: Von Neumann vs. Harvard

8 P

IN EINEM SATZ Beim Von-Neumann-Rechner teilen sich Programm und Daten einen Speicher und eine Leitung, bei Harvard hat jedes seinen eigenen Speicher und Bus.

STELL DIR VOR Eine **Küche mit nur einer Tür**: Der Koch holt durch dieselbe Tür erst das Rezept, dann die Zutaten, nie beides gleichzeitig (Von-Neumann-Flaschenhals). Mit **zwei Türen** (Harvard) geht es schneller, der Umbau ist aber teurer. Der **Befehlszähler** ist der Finger auf der aktuellen Rezeptzeile.

Größter Einzelblock. Laut VL05: „**Skizzen, Eigenschaften, Pro und Con**“. Übe die beiden Skizzen so oft, bis du sie in 2 Minuten sauber zeichnen kannst.

Von-Neumann-Architektur („Princeton-Architektur“) Rückschau

Idee: ein **Universalrechner**, der **nicht an ein festes Programm gebunden** ist und über Ein-/Ausgabegeräte verfügt; 1946 von John von Neumann beschrieben. Im Von-Neumann-Rechner werden **Daten und Programme binär kodiert und liegen im gleichen Speicher**: Es gibt **nur einen gemeinsamen Speicher**, und Programmcode und Daten werden **über den gleichen Bus** transportiert. Programme können daher wie Daten geladen und geändert werden (Stored-Program-Prinzip, *Begriff = Ergänzung*). Von Neumanns Verdienste: erste wissenschaftlich-mathematische Beschäftigung mit der Konstruktion von Rechenmaschinen und das **sequentielle Prinzip** (Von-Neumann-Zyklus).

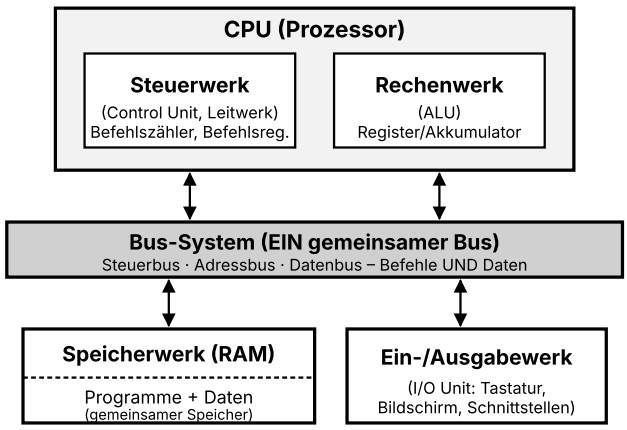


Abb. 9.1 – Von-Neumann-Rechner: 5 Komponenten, ein Speicher, ein Bus

Komponente	Aufgabe
Rechenwerk (ALU, Arithmetic Logic Unit)	führt Rechenoperationen und boolesche Verknüpfungen aus
Steuerwerk (Control Unit, Leitwerk)	interpretiert die Befehle des Programms, verschaltet Datenquelle, Datensenke und ALU-Komponenten, regelt die Befehlsfolge
Bus-System	Kommunikation zwischen den Komponenten (Steuer-, Adress-, Datenbus)
Speicherwerk (Memory, RAM)	speichert **sowohl Programme als auch Daten**
Ein-/Ausgabewerk (I/O Unit)	Ein- und Ausgabe zum Anwender (Tastatur, Bildschirm) oder zu anderen Systemen (Schnittstellen)

Steuerwerk + Rechenwerk (+ Register) bilden zusammen die **CPU**.

Von-Neumann-Zyklus – 5 Phasen (auf der Folie mit „!“ markiert) Rückschau

Sequentielle Arbeitsweise: Jede Phase kann mehrere Takte dauern; Prozessor und Speicher kommunizieren direkt über **einen** Bus, über den Befehle **und** Daten transportiert werden.

#	Phase	Was passiert?
1	**FETCH**	abzuarbeitenden Befehl aus dem Speicher in das **Befehlsregister** der CPU laden
2	**DECODE**	Steuerwerk löst den Befehl in Schaltinstruktionen für das Rechenwerk auf
3	**FETCH OPERANDS**	Parameter (Operanden) für den Befehl aus dem Speicher holen
4	**EXECUTE**	Rechenwerk führt die Operation aus, ggf. Ergebnis in den Speicher schreiben
5	**UPDATE INSTRUCTION POINTER**	**Befehlszähler** wird erhöht bzw. bei Sprüngen entsprechend gesetzt; dann beginnt der Zyklus von vorne

Beispiel *(Ergänzung, Bezug zum 6502-Assembler aus VL02)*: `ADC NUM2` – FETCH holt den Befehl von der Adresse im Befehlszähler, DECODE erkennt „addiere zum Akkumulator“, FETCH OPERANDS holt den Wert von NUM2 aus dem Speicher, EXECUTE rechnet in der ALU, dann zeigt der Befehlszähler auf den nächsten Befehl.

Harvard-Architektur Rückschau

Entwickelt, um den **Engpass der Von-Neumann-Architektur zu überwinden** (Ursprung Harvard Mark I, 1944).

- **Separate Speicher**: getrennte Speicherbänke mit separaten physischen Adressen für **Befehle** und **Daten**.
- **Separate Busse** für Befehle und Daten.
- **Ausführung**: „Einzelzyklus“ – Befehl und Daten können **in einem Schritt** gleichzeitig über die verschiedenen Busse

geholt werden → **schneller**.

- **Kosten**: in der Regel **teurer**.
- **Verwendung**: Mikrocontroller, Signalverarbeitung (DSP), spezielle Rechner für maschinelles Lernen/neuronale Netze, zunehmend auch PCs/Server.

(nur Folie) Prozessorbeispiele: Intel x86 = Von Neumann; AVR (Arduino) = Harvard; ARM und Apple M = **modifizierte Harvard-Architektur** (getrennte Busse/Caches, gemeinsamer Hauptspeicher).

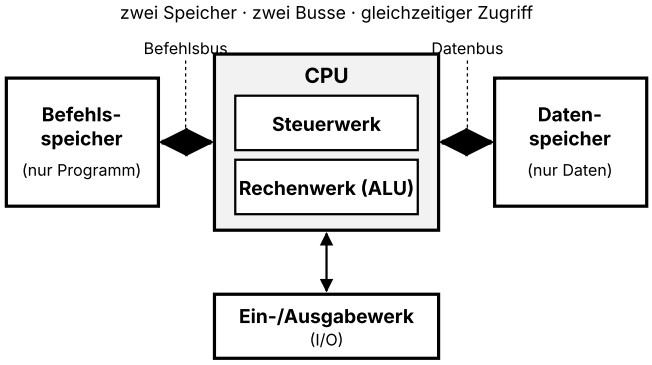


Abb. 9.2 – Harvard-Rechner: getrennte Speicher und Busse für Befehle und Daten

Vergleich und Bewertung (Pro und Contra)

Kriterium	Von-Neumann	Harvard
Speicher	**ein gemeinsamer** Speicher für Programme und Daten	**getrennte** Speicher (eigene Adressräume) für Befehle und Daten
Bus	**ein gemeinsamer** Bus für Befehle und Daten	**getrennte** Busse für Befehle und Daten
Ablauf	Befehl und Daten **nacheinander** über den Bus → mind. 2 Taktzyklen pro Anweisung	Befehl und Daten **gleichzeitig** in einem Schritt („Einzelzyklus“)
Geschwindigkeit	begrenzt durch den **Von-Neumann-Flaschenhals** (Bottleneck)	**schneller** – vermeidet den Bottleneck
Kosten/Aufwand	**Pro:** einfachere Implementierung, **geringere Kosten**	**Contra:** in der Regel **teurer** (doppelte Speicher-/Busstruktur)
Flexibilität *(Erg.)*	Speicher frei zwischen Programm und Daten aufteilbar; Programme zur Laufzeit ladbar	feste Aufteilung der Speichergrößen
Sicherheit	**Contra:** Angriffsvektoren, z. B. **Buffer-Überlauf** bzw. Einschleusen von als Daten getarntem Code	Code und Daten physisch getrennt → dieser Angriff ist erschwert *(Erg.)*
Einsatz	klassische PCs/Universalrechner (x86)	Mikrocontroller, Signalverarbeitung, KI-Spezialrechner, zunehmend PCs/Server

> **Musterantwort „Erläutern Sie den Unterschied“ (≈ 4 Sätze):** Beim Von-Neumann-Rechner liegen Programme und Daten binär kodiert in einem gemeinsamen Speicher und werden über einen gemeinsamen Bus transportiert. Deshalb müssen Befehl und Daten nacheinander geholt werden, was als Von-Neumann-Flaschenhals die Geschwindigkeit begrenzt; zudem kann als Daten getarnter Code eingeschleust werden (Buffer Overflow). Die Harvard-Architektur trennt Befehls- und Datenspeicher physisch und nutzt getrennte Busse, sodass Befehl und Daten in einem Schritt gleichzeitig geladen werden können. Sie ist dadurch schneller, aber aufwendiger und teurer, während Von-Neumann einfacher und günstiger zu realisieren ist.

Komponenten im Detail: CPU, Register, Bussystem (Rückschau)

- **Steuerwerk**: holt Befehle aus dem Hauptspeicher, bestimmt den Befehlstyp, lädt Operanden in Register, lässt das Rechenwerk rechnen und entscheidet (ggf. abhängig vom Ergebnis), wo es weitergeht. **Rechenwerk**: arithmetische und logische Operationen, teils an Coprozessoren ausgelagert.
- **Register**: kleine Hochgeschwindigkeitsspeicher in der CPU für je einen Wert; Datenschnittstelle zum Rechenwerk. Wichtigstes Register ist der **Befehlszähler** (Program Counter: **Adresse** des nächsten Befehls), dann das **Befehlsregister** (der **aktuelle Befehl** selbst).
- **Bussystem** – Lesen: Lesebefehl über den **Steuerbus**, Adresse der Speicherzelle über den **Adressbus**, Daten über den **Datenbus** zur CPU. Schreiben: Schreibbefehl über den Steuerbus, Adresse über den Adressbus, Daten über den Datenbus zur Zelle. Adress- und Datenbus arbeiten **parallel** (viele Leitungen gleichzeitig, z. B. Apple M1: 64-Bit-Adressbus, 128-Bit-Datenbus); Peripherie meist **seriell** (USB = Universal Serial Bus).

> **Folienfehler/-ungenauigkeit (Rückschau VL03a F. 3):** „Wesentliche Ideen … 1937 in der Zuse Z1 realisiert“ ist umstritten: Die Z1 las ihr Programm von Lochstreifen, ein gemeinsamer Programm-/Datenspeicher (Stored Program) fehlte. Für die Klausur: Von-Neumann-Architektur = 1946, John von Neumann.

Exkurs KI-Hardware (VL03c, vermutlich nicht klausurrelevant): 3D-Grafik und neuronale Netze bestehen hauptsächlich aus Matrix-/Vektoroperationen (Skalarprodukte) → GPUs als KI-Beschleuniger; Engpass ist die Speicherbandbreite (HBM, Unified Memory); spezialisierte Chips: NPU (PC/Smartphone), TPU (Rechenzentrum).

⚠ TYPISCHE KLAUSURFALLEN

- Von Neumann = gemeinsamer Speicher **und** gemeinsamer Bus. Harvard = getrennte Speicher **und** getrennte Busse. Beides nennen!
- Es sind **fünf** Komponenten (Rechenwerk, Steuerwerk, Bus, Speicherwerk, E/A-Werk) – der Bus wird gern vergessen.
- Der Zyklus hat **fünf** Phasen, nicht nur Fetch – Decode – Execute. Reihenfolge: Fetch, Decode, Fetch Operands, Execute, Update Instruction Pointer.
- **Befehlszähler** = Adresse des *nächsten* Befehls; **Befehlsregister** = *aktueller* Befehl.
- Harvard ist schneller, aber **teurer**; Von Neumann einfacher und **billiger** – Pro/Contra immer paarweise.
- Der Flaschenhals ist der **Bus** (Befehle und Daten nacheinander), nicht die ALU.
- Skizze: alle Kästen **beschriften**, Pfeile in beide Richtungen, bei Harvard **zwei** getrennte Speicher und Busse zeichnen.

DAS MUSST DU KÖNNEN

- ☐ Von-Neumann-Skizze aus dem Kopf
- ☐ Harvard-Skizze aus dem Kopf
- ☐ 5 Komponenten nennen und erläutern
- ☐ 5 Phasen des Zyklus in Reihenfolge
- ☐ Flaschenhals erklären
- ☐ Sicherheitsproblem (Buffer Overflow)
- ☐ Pro/Contra beider Architekturen
- ☐ Einsatzgebiete Harvard nennen
- ☐ Steuer-/Adress-/Datenbus beim Lesen/Schreiben
- ☐ Befehlszähler vs. Befehlsregister

10 · Speicher: Arten, Hierarchie, RAID

4 P

IN EINEM SATZ Speicher ist umso schneller und teurer, je näher er an der CPU sitzt; RAID verteilt Daten so auf mehrere Platten, dass es schneller geht oder ein Ausfall nichts kaputt macht.

STELL DIR VOR **Cache = Schreibtisch statt Keller**: Die Bücher für heute liegen auf dem Tisch, der Rest im Keller (RAM). Holst du eins aus dem Keller, nimmst du die Nachbarn gleich mit (Lokalität); fehlt doch eins, läufst du runter (Cache-Miss). **RAID 0** = zwei Leute tragen abwechselnd Umzugskisten (schnell, aber stolpert einer, ist seine Hälfte weg), **RAID 1** = Mitschrift für die Freundin kopieren, **RAID 5** = Prüfsumme pro Zeile.

Speicherarten Rückschau

Art	Eigenschaften
Register	in der CPU, extrem schnell, sehr klein, teuer
Cache (L1, L2, L3)	sehr schneller Zwischenspeicher in/vor der CPU, begrenzte Größe
Hauptspeicher / RAM	Programme und Daten, die nach dem Start erst geladen werden; bei Von Neumann gemeinsam, bei Harvard getrennt adressiert; **flüchtig**
Flash (früher ROM/EPROM)	meist nur gelesen (BIOS, Firmware); behält den Inhalt **ohne Strom**

Cache und Lokalität Rückschau

Problem: Die CPU wird immer schneller als der Hauptspeicher, den ganzen Speicher auf die CPU zu bauen ist zu teuer. Kompromiss: ein kleiner, schneller **Cache**. Er funktioniert wegen der **Lokalität**: Programme greifen in kurzer Zeit nur auf einen kleinen Teil des Speichers zu. Deshalb lädt man beim Zugriff auf ein Wort auch seine **Nachbarn** in den Cache. Beispiele: **Schleifen** (derselbe Code mehrfach) und **Arrays/Tabellen** (benachbarte Datensätze). Liegen Daten wider Erwarten nicht im Cache, müssen sie über den Bus nachgeladen werden (Cache-Miss).

Arbeitsspeicher und Adressbusbreite Rückschau

Der Arbeitsspeicher besteht aus Zellen mit je einer **Adresse**; kleinste adressierbare Einheit ist das **Byte**. Mit **n** Adressleitungen kann man $\mathbf{2^n}$ Zellen (Bytes) adressieren.

```
16 Leitungen → 2¹⁶ = 65.536 Byte = 64 KB   (z. B. 6502, 1975)
20 Leitungen → 2²⁰ = 1.048.576 Byte = 1 MB
32 Leitungen → 2³² Byte = 4 GB          (umgekehrt: 4 GB = 2² · 2³⁰ = 2³² → 32 Leitungen)
512 GB = 2⁹ · 2³⁰ = 2³⁹ Byte → 39 Adressleitungen
48 Leitungen → 2⁴⁸ Byte = 256 TB        (laut Folie Intel Core i7, 14. Gen.)
```

Speicherhierarchie Rückschau

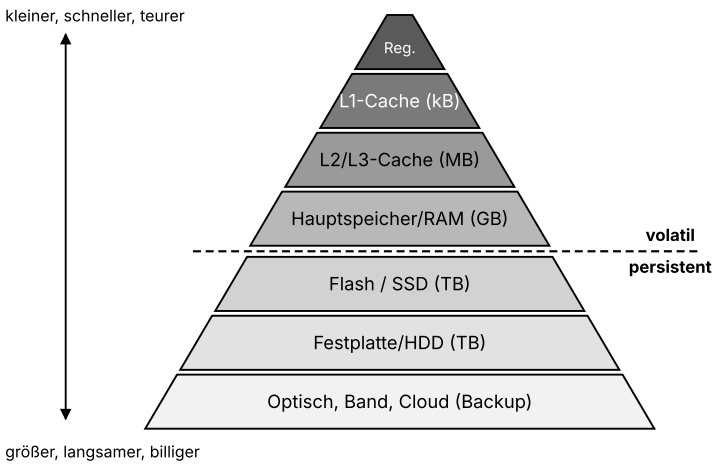


Abb. 10.1 – Speicherhierarchie (nach VL03b-Rückschau F. 9); L1 im Kern, L2 außerhalb des Kerns, L3 zwischen den Kernen

Je weiter unten in der Pyramide, desto **größer** werden **Zugriffszeit** (langsamer), **Kapazität** und **Bits pro Euro** (billiger pro Bit). Ganz oben: klein, schnell, teuer.

Die gestrichelte Linie trennt **volatile** (flüchtige: Inhalt geht ohne Strom verloren – Register, Cache, RAM) von **persistenten** Speichern (Inhalt bleibt erhalten – Flash/SSD, Festplatte, Band).

nur Folie Festplatte: magnetische Scheiben, schwebender Lesekopf; SSD: Flash ohne bewegliche Teile, begrenzte Schreibzyklen; LTO-Band für Backup/Archiv.

RAID – Redundant Array of Independent Disks Rückschau

Mehrere Festplatten erscheinen dem System als **ein logisches Laufwerk**. Ziel: höherer **Durchsatz** und/oder **Ausfallsicherheit**. NAS-Systeme lassen sich für verschiedene RAID-Level konfigurieren. Die „gebräuchlichsten" Level laut Folie sind 0, 1 und 5 (dazu 6):

Level	Name / Prinzip	min. Platten	Ausfall verkraftet	Nutzkapazität (n Platten à C)	4 × 2 TB	5 × 4 TB
RAID 0	**Striping**: Datenblöcke („Strips“) reihum auf alle Platten → schnell, **keine Redundanz**	2 *(Erg.)*	**0**	n · C	8 TB	20 TB
RAID 1	**Mirroring**: jede Platte wird gespiegelt	2 *(Erg.)*	1 je Spiegelpaar	n · C / 2	4 TB	(gerade Anzahl nötig)
RAID 5	**Leistung und Parität**: Striping + Paritätsblock, reihum über alle Platten verteilt	**3**	**1**	(n − 1) · C	6 TB	16 TB
RAID 6	wie RAID 5, aber doppelte Paritätsinformation	**4**	**2**	(n − 2) · C	4 TB	12 TB

Layout RAID 5 mit 4 Platten (P = Parität, rotiert):

Platte 1	Platte 2	Platte 3	Platte 4
Strip 0	Strip 1	Strip 2	P 0–2
Strip 3	Strip 4	P 3–5	Strip 5
Strip 6	P 6–8	Strip 7	Strip 8

Wie hilft Parität? *(Ergänzung)* Der Paritätsblock ist das XOR der Datenblöcke derselben Zeile. Beispiel: 1011 XOR 0110 XOR 1100 = **0001**. Fällt die Platte mit 0110 aus, rechnet man 1011 XOR 1100 XOR 0001 = **0110** zurück. **RAID ersetzt kein Backup** (versehentliches Löschen wird mitgespiegelt).

Folienungenauigkeiten (VL03b-Rückschau): F. 9 nennt Flash „local primary storage“ – üblich ist: **Primärspeicher = Arbeitsspeicher**, SSD/HDD = Sekundär-/Massenspeicher. F. 4 „Cache-fault“ – Fachbegriff ist **Cache-Miss**. F. 10 „Bei RAID 2 & 3 … Ebene der einzelnen Bits“ – nur RAID 2 arbeitet bitweise, RAID 3 byteweise. F. 11: Das Bild zu RAID 1 zeigt gestreifte *und* gespiegelte Platten (eigentlich RAID 10); für die Klausur gilt RAID 1 = Spiegelung. F. 5 vs. F. 6: 64-Bit- bzw. 48-Bit-Adressbus widersprechen sich (real nutzen CPUs weniger als 64 physische Adressbits).

⚠ TYPISCHE KLAUSURFALLEN

- Nach unten in der Hierarchie: **langsamer, größer, billiger pro Bit**. „Zugriffszeit steigt“ = langsamer.
- RAM ist **volatil**, Flash/SSD ist **persistent**.
- Adressbus: $\mathbf{2^n}$ Bytes, nicht n · 8 oder 2 · n. 1 KB = 2^{10}, 1 MB = 2^{20}, 1 GB = 2^{30} Byte.
- **RAID 0 hat keine Redundanz** – trotz „R“ im Namen.
- RAID 5: **mindestens 3** Platten, **1** Ausfall; RAID 6: **mindestens 4** Platten, **2** Ausfälle. RAID 1: halbe Kapazität.
- Cache lohnt sich wegen **Lokalität** (Schleifen, Arrays), nicht weil er groß ist.

DAS MUSST DU KÖNNEN

- ☐ Speicherarten nennen und einordnen
- ☐ Hierarchie-Pyramide skizzieren
- ☐ volatil/persistent zuordnen
- ☐ Cache und Lokalität erläutern
- ☐ Adressleitungen ↔ Speichergröße rechnen
- ☐ RAID 0/1/5/6: Prinzip, Mindestplatten, Ausfälle
- ☐ Nutzkapazität berechnen

11 · Datenbanken und SQL

4–6 P

IN EINEM SATZ Eine Datenbank speichert jede Information nur einmal in Tabellen, und eine SQL-Abfrage ist dasselbe wie in Excel eine Liste filtern und Spalten ausblenden.

STELL DIR VOR **SQL = Excel-Filter**: Du öffnest die Liste (FROM), setzt den Filter „Abteilung = 5“ (WHERE), blendest alle Spalten außer Name und Gehalt aus (SELECT); GROUP BY ist eine Pivot-Tabelle „pro Abteilung die Summe“. Das **DBMS** ist die Bibliothekarin, das **Data Dictionary** ihr Katalog; der **Fremdschlüssel** ist wie deine Versichertennummer auf jeder Arztrechnung.

Laut VL05: „Eigenschaften, **Analyse** SQL, **Synthese** SQL“ sowie „Modellierung einfacher Entitäten, **Dictionary**“. Du musst also ein SQL-Statement anhand vorgegebener Tabellen „ausführen“ und selbst einfache Abfragen schreiben können.

Vom Dateiansatz zum DBMS (Rückschau)

Bis in die 1970er stellte das Betriebssystem **Dateien** bereit; jede Abteilung (Buchhaltung, Personal, Vertrieb …) baute eigene Anwendungen mit eigenen Dateien. **Probleme des Dateiansatzes:**

- **Redundanz und Inkonsistenz**: dieselben Daten mehrfach gespeichert → widersprüchliche Kopien (Adresse nur in der Buchhaltung geändert).
- **Abhängigkeit zwischen Programm und Daten**: ändert sich die Datenstruktur, muss das Programm geändert werden.
- **Fehlender Datenaustausch**, mangelnde Flexibilität bei abteilungsübergreifenden Prozessen.
- **Mangelnde Datensicherheit**: keine stringente Verwaltung von Zugriffsrechten.

> **Datenbank (DB)**: Sammlung logisch zusammenhängender Daten mit bestimmter Bedeutung, für einen bestimmten Zweck entworfen. **Datenbankmanagementsystem (DBMS)**: Software, mit der Benutzer eine Datenbank anlegen, pflegen und abfragen. **Datenbanksystem = DB + DBMS** (+ Anwendungen). Das DBMS verwaltet zwei Bestände: **Metadaten im Data Dictionary** (Datenkatalog: welche Tabellen, Attribute, Datentypen, Einschränkungen) und die **Nutzdaten** (Kunden, Lieferanten …).

Merkmale des Datenbankansatzes: Datenkatalog (Data Dictionary) mit Metadaten · Datenisolation/-abstraktion (interne Speicherung verborgen) · mehrere Sichten (Views) · Mehrbenutzerzugriff und -kontrolle.

Nachteile: hohe Anfangsinvestition, für einfache Fälle zu komplex, Mehraufwand für Sicherheit, Nebenläufigkeit, Recovery. Dateien bleiben sinnvoll bei einfachen, stabilen Daten, harten Echtzeitanforderungen oder ohne Mehrbenutzerzugriff.

Vorteile eines DBMS: Redundanzkontrolle · Zugriffsbeschränkungen (Autorisierung) · persistente Speicherung · Ableitungen anhand von Regeln · Mehrbenutzerschnittstellen (grafisch, textuell, ODBC) · Beziehungen zwischen Daten werden überwacht · Integritätsbedingungen (z. B. gültige Punkteskala) · Recovery nach Fehlern (Transaktionen).

Datenmodelle und ER-Modell (Rückschau)

Ebene	Beschreibt	Beispiel
konzeptuell	die reale Welt aus Benutzersicht, **unabhängig vom DBMS**	ER-Modell
logisch	für eine **Klasse** von DBMS, nah an der Speicherung	relationales Modell (Tabellen)
physisch	Details der Speicherung auf dem Rechner	Dateien, Indizes

Das **Entity-Relationship-Modell** (Peter Chen, 1976) ist der wichtigste Ansatz der konzeptuellen Modellierung. Begriffe: **Entität** = „Phänomen“ (Person, Ort, Ding, Ereignis), über das Daten gespeichert werden; **Entitätstyp** = Abstraktion gleichartiger Entitäten mit gemeinsamen Attributen (ANGESTELLTER); **Entitätsmenge** = alle Entitäten eines Typs; **Attribut** = Eigenschaft (Name, Alter); **Schlüsselattribut** = eindeutig für jede Entität (unterstrichen), ggf. zusammengesetzt (Kennzeichen + Land); **Beziehungstyp** = Zusammenhang zwischen Entitätstypen (kann eigene Attribute haben); **Grad** = Anzahl beteiligter Entitätstypen (binär = 2, ternär = 3).

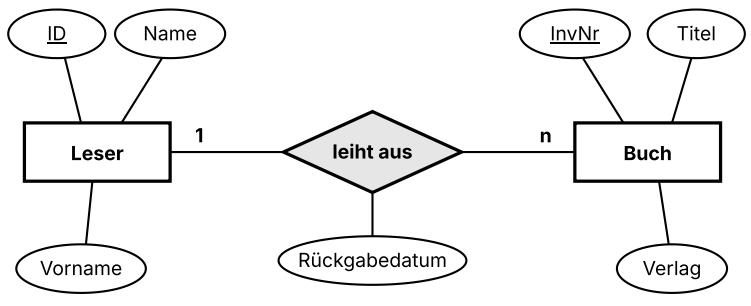


Abb. 11.1 – ER-Diagramm (Chen-Notation) nach dem Folienbeispiel „Bibliothek“: Rechteck = Entitätstyp, Raute = Beziehungstyp, Ellipse = Attribut, unterstrichen = Schlüssel

Lesart: Ein Leser kann n Bücher ausleihen; ein Buch ist zu einem Zeitpunkt an höchstens einen Leser verliehen. Das **Rückgabedatum** gehört weder zum Buch noch zum Leser, sondern zur **Beziehung**.

Kardinalitäten (an wie vielen Beziehungen nimmt eine Entität teil?):
1:1 Leser – Bibliothekskarte
1:n Rechnung – Posten
m:n Leser bestellt Buch; Student besucht Vorlesung

Weitere Chen-Symbole (Rückschau): doppeltes Rechteck = schwache Entität; doppelte Ellipse = mehrwertiges Attribut; gestrichelte Ellipse = abgeleitetes Attribut; Doppellinie = totale Teilnahme.

Relationales Modell Rückschau

Vom IBM-Forscher **Edgar F. Codd** 1970 entwickelt, heute das verbreitetste logische Modell (Oracle, IBM DB2, MS SQL Server, MySQL …). Eine Datenbank ist eine Sammlung von **Relationen = Tabellen**. Eine **Zeile** heißt **Tupel** (Datensatz), eine **Spalte** heißt **Attribut**. Jedes Attribut hat einen **Wertebereich (Domäne)**, und jeder Wert ist **atomar** (nicht weiter zerlegbar). `null` = Wert unbekannt/nicht vorhanden. Beziehungen werden über **Fremdschlüssel** abgebildet: ein Attribut, das auf den (Primär-)Schlüssel einer anderen Tabelle verweist. Eine **m:n**-Beziehung braucht eine eigene **Hilfstabelle** mit beiden Fremdschlüsseln.

ER-Element	wird im relationalen Modell zu … *(Faustregeln aus den Folienbeispielen)*
Entitätstyp	eigene Tabelle; Schlüsselattribut → Primärschlüssel
1:n-Beziehung	Fremdschlüssel auf der **n-Seite** (PROJEKT.ABTNR → ABTEILUNG)
1:1-Beziehung	Fremdschlüssel auf einer der beiden Seiten (ABTEILUNG.MGRSSN → ANGESTELLTER)
m:n-Beziehung	Hilfstabelle mit beiden Fremdschlüsseln als zusammengesetztem Schlüssel + Beziehungsattributen: ARBEITET_AN(ESSN, PNR, STUNDEN)

SQL – Structured Query Language Rückschau

In den 1970ern bei IBM entwickelt, seit 1987 ISO-Standard, **deklarativ** (man beschreibt, *was* man will, nicht *wie*). Teile: **Data Definition** (CREATE: Schema anlegen), **Data Manipulation** (INSERT, UPDATE, DELETE), **Data Query** (SELECT).

```
CREATE TABLE Firma
( Kurzname       char(15) not null,
  Name           char(40),
  Adresse        char(60),
  Kurzmitteilung char(400) );

INSERT INTO Student (MatrNr, Vorname, Name, Semester)
VALUES (12345678, 'Peter', 'Hinz', 4),
       (10101010, 'Petra', 'Kunz', 2);
```

```
UPDATE Student
SET    Exmatrikuliert = TRUE
WHERE  Semester > 99;

DELETE FROM Student
WHERE  MatrNr = 87654321;

-- Grundstruktur einer Abfrage:
SELECT <Attributliste>   -- welche Spalten
FROM   <Tabellenliste>   -- aus welchen Tabellen
WHERE  <Bedingung>;      -- welche Zeilen
```

`char(n)` = Text fester Länge, `not null` = Pflichtfeld. Ohne `WHERE` wirken UPDATE und DELETE auf **alle** Zeilen. Aggregatfunktionen fassen mehrere Zeilen zu einem Wert zusammen: **SUM, MAX, MIN, AVG, COUNT**; **GROUP BY** liefert eine Ergebniszeile pro Gruppe.

Beispieldaten (Auszug „Firma“, Folie)

ANGESTELLTER (Primärschlüssel SSN, Fremdschlüssel ABT → ABTEILUNG)

VNAME	NNAME	SSN	GDATUM	GEHALT	ABT
John	Smith	123456789	1965-01-09	30000	5
Franklin	Wong	333445555	1955-12-08	40000	5
Alicia	Zelaya	999887777	1968-07-19	25000	4
Jennifer	Wallace	987654321	1941-06-20	43000	4
Ramesh	Narayan	666884444	1962-09-15	38000	5
Joyce	English	453453453	1972-07-31	25000	5
Ahmad	Jabbar	987987987	1969-03-29	25000	4
James	Borg	888665555	1937-11-10	55000	1

ABTEILUNG (PS ABTNUMMER)

ANAME	ABTNUMMER	MGRSSN
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

PROJEKT (PS PNUMMER)

PNUMMER	PSTANDORT	ABTNR
1	Bellaire	5
2	Sugarland	5
3	Houston	5
10	Stafford	4
20	Houston	1
30	Stafford	4

Analyse: Was liefert die Abfrage? (alle Ergebnisse mit SQLite nachgeprüft)

Vorgehen bei der Analyse: (1) **FROM** – welche Tabellen? Bei mehreren: alle Kombinationen, die die Join-Bedingung erfüllen. (2) **WHERE** – Zeilen filtern. (3) **GROUP BY** – Gruppen bilden. (4) **SELECT** – Spalten bzw. Aggregate ausgeben.

Abfrage	Ergebnis

`SELECT VNAME, NNAME, GEHALT`
`FROM ANGESTELLTER`
`WHERE ABT = 5 AND GEHALT > 30000;`
	Franklin Wong 40000
	Ramesh Narayan 38000
	(Smith hat genau 30000 → nicht > 30000!)

`SELECT SUM(GEHALT), MAX(GEHALT),`
`MIN(GEHALT), AVG(GEHALT)`
`FROM ANGESTELLTER;`
	281000 | 55000 | 25000 | 35125
	(eine einzige Ergebniszeile, Folienbeispiel)

`SELECT ABT, COUNT(*), AVG(GEHALT)`
`FROM ANGESTELLTER`
`GROUP BY ABT;`
	5 | 4 | 33250
	4 | 3 | 31000
	1 | 1 | 55000

`SELECT PNUMMER, ABTNR, NNAME, GDATUM`
`FROM PROJEKT, ABTEILUNG, ANGESTELLTER`
`WHERE ABTNR = ABTNUMMER`
`AND MGRSSN = SSN`
`AND PSTANDORT = 'Stafford';`
	10 | 4 | Wallace | 1941-06-20
	30 | 4 | Wallace | 1941-06-20
	Join: Projekt → kontrollierende Abteilung → deren Managerin
	(nur Folie)

`SELECT GDATUM, ADRESSE FROM ANGESTELLTER`
`WHERE VNAME = 'John' AND INITIAL = 'B'`
`AND NNAME = 'Smith';`
	1965-01-09 | 731 Fondren, Houston, TX *(Folienbeispiel; die Spalten INITIAL und ADRESSE stehen in der Folientabelle, sind oben aus Platzgründen weggelassen)*

Synthese: Abfrage selbst schreiben – Rezept

1. Welche **Spalten** sollen ausgegeben werden? → `SELECT` (bei „wie viele“, „Summe“, „Durchschnitt“ → Aggregatfunktion).
2. In welchen **Tabellen** stehen diese Spalten und die Bedingungen? → `FROM`; bei mehreren Tabellen die **Join-Bedingung** Fremdschlüssel = Primärschlüssel in WHERE.
3. Welche **Zeilen**? → `WHERE` mit `AND/OR`, Texte in **geraden einfachen Hochkommas** `'…'`.
4. „pro …“/„je …“ → `GROUP BY` über diese Spalte.

„Wie viele Projekte kontrolliert jede Abteilung?“ → `SELECT ABTNR, COUNT(*) FROM PROJEKT GROUP BY ABTNR;`
Ergebnis: 1 | 1, 4 | 2, 5 | 3

„Namen aller Angestellten mit Gehalt ≥ 40000“ → `SELECT NNAME FROM ANGESTELLTER WHERE GEHALT >= 40000;`
Ergebnis: Wong, Wallace, Borg

„Erhöhe die Gehälter der Abteilung 4 um 10 %“ → `UPDATE ANGESTELLTER SET GEHALT = GEHALT * 1.1 WHERE ABT = 4;`

Folienbeispiel GROUP BY: `SELECT Kunden.KNr, COUNT(Bestellungen.BestNr) AS Anzahl FROM Kunden, Bestellungen WHERE Kunden.Land='CH' AND Kunden.KNr=Bestellungen.KNr GROUP BY Kunden.KNr` zählt die Bestellungen je Schweizer Kunde. Hinweis der Folie: **Kunden ohne Bestellungen werden nicht ausgegeben** (die Join-Bedingung ist für sie nie erfüllt).

▶ REZEPT: SQL-ABFRAGE VON HAND AUSFÜHREN – SELECT, WHERE, JOIN, GROUP BY AN EINER KLEINEN TABELLE

Reihenfolge beim **Ausführen** (nicht beim Schreiben!): ① **FROM/JOIN** Tabellen verbinden → ② **WHERE** Zeilen streichen → ③ **GROUP BY** übrige Zeilen in Gruppen sortieren → ④ **SELECT** Spalten bzw. ein Aggregat pro Gruppe ausgeben.
Zwischenergebnis ruhig als Tabelle auf das Schmierblatt schreiben. *(Eigenes Beispiel, alle Ergebnisse mit SQLite geprüft.)*

KUNDE (PS KNR)

KNR	NAME	ORT
1	Anna	Köln
2	Ben	Berlin
3	Cem	Köln
4	Dana	Ulm

BESTELLUNG (PS BNR, FS KNR → KUNDE)

BNR	KNR	BETRAG
101	1	40
102	2	25
103	1	15
104	3	60
105	2	30
106	1	20

① Zwischenergebnis JOIN über B.KNR = K.KNR

BNR	NAME	ORT	BETRAG
101	Anna	Köln	40
102	Ben	Berlin	25
103	Anna	Köln	15
104	Cem	Köln	60
105	Ben	Berlin	30
106	Anna	Köln	20

Dana fehlt: keine Bestellung, kein Partner

Abfrage	Schritte	Ergebnis
`SELECT NAME FROM KUNDE` `WHERE ORT = 'Köln';`	① nur KUNDE · ② Zeilen mit ORT ≠ Köln streichen (Ben, Dana) · ④ nur Spalte NAME	Anna Cem

`SELECT B.BNR, K.NAME, B.BETRAG`
`FROM BESTELLUNG B JOIN KUNDE K`
`ON B.KNR = K.KNR`
`WHERE B.BETRAG >= 25;`
	① Join-Tabelle oben (6 Zeilen) · ② 103 (15) und 106 (20) streichen, 102 mit genau 25 **bleibt** (>=) · ④ drei Spalten	101 | Anna | 40
		102 | Ben | 25
		104 | Cem | 60
		105 | Ben | 30

`SELECT K.ORT, COUNT(*), SUM(B.BETRAG)`
`FROM KUNDE K JOIN BESTELLUNG B`
`ON K.KNR = B.KNR`
`GROUP BY K.ORT;`
	① Join-Tabelle · ③ Gruppe Berlin {25, 30}, Gruppe Köln {40, 15, 60, 20} · ④ je Gruppe **eine** Zeile: Anzahl und Summe	Berlin | 2 | 55
		Köln | 4 | 135

`SELECT KNR, COUNT(*) FROM BESTELLUNG`
`WHERE BETRAG > 20`
`GROUP BY KNR;`
	② **vor** dem Gruppieren 103 (15) und 106 (20) streichen · ③ KNR 1 {101}, KNR 2 {102, 105}, KNR 3 {104}	1 | 1
		2 | 2
		3 | 1

Komma-Schreibweise der Folien = JOIN … ON: `FROM BESTELLUNG B, KUNDE K WHERE B.KNR = K.KNR AND B.BETRAG >= 25` liefert genau dasselbe. **Selbst schreiben**: Rezept oben rückwärts – Spalten (SELECT) → Tabellen + Join-Bedingung FS = PS (FROM) → Filter (WHERE) → „pro …“ (GROUP BY).

OLTP und OLAP (Rückschau)

	OLTP – Online Transaction Processing	OLAP – Online Analytical Processing
Zweck	operatives Tagesgeschäft, Transaktionen (Bestellung, Buchung)	Analyse: BI, Data Warehouse
Daten	**aktuelle** Daten	inkl. **Historie**
Zugriff	**Lesen und Schreiben**, ganze Datensätze	(fast) **nur Lesen** („read only“ Analytics)
Speicherung	**Row Store** – zeilenweise Ablage („klassisch“)	**Column Store** – spaltenweise Ablage; schnell, wenn nur wenige Spalten gebraucht werden; gut komprimierbar → oft komplett im Hauptspeicher (**In-Memory-DB**)
Beispiele	Oracle, MySQL	Amazon Redshift, SAP HANA

Daten wandern per **ETL** (Extract, Transform, Load) vom OLTP-System ins Data Warehouse. Beispiel (Country, Product, Sales) = (US, Alpha, 3000), (US, Beta, 1250), (JP, Alpha, 700), (UK, Alpha, 450): Row Store speichert `US Alpha 3000 | US Beta 1250 | …`, Column Store `US US JP UK | Alpha Beta Alpha Alpha | 3000 1250 700 450`. Für den Gesamtumsatz (5400) muss der Column Store nur die Sales-Spalte lesen.

Folienfehler (VL04 bzw. Rückschau): „tertiär“ (3 Entitätstypen) heißt richtig **ternär**. Beziehungstypen werden im Firmenbeispiel „Relationen“ genannt – im relationalen Modell ist eine **Relation aber eine Tabelle**. Im INSERT-Beispiel enden `'Hinz‘` und `'Kunz‘` mit typografischem Anführungszeichen → in SQL nur gerade Apostrophe `'…'`. `DELETE Student WHERE …` – Standard-SQL verlangt `DELETE FROM`. Im SELECT-Beispiel heißt die Spalte `NNAME`, nicht `NAME`. Kommentare mit `//` (Join-Folie) sind kein SQL – richtig `--`. „Tedd Codd“ → Edgar F. „Ted“ Codd. ARBEITET_AN ist insgesamt **m:n**, nicht „1:N / 1:N“.

⚠ TYPISCHE KLAUSURFALLEN

- **Relation = Tabelle**, nicht Beziehung. **Tupel = Zeile**, **Attribut = Spalte**.
- DB ≠ DBMS ≠ Datenbanksystem (DB + DBMS). Data Dictionary = **Metadaten**, nicht Nutzdaten.
- Fremdschlüssel bei 1:n auf die **n-Seite**; m:n braucht **immer** eine Hilfstabelle.
- Bei der Analyse auf **>** vs. **>=** achten (Smith mit genau 30000!) und auf AND vs. OR.
- Aggregat ohne GROUP BY → genau **eine** Ergebniszeile. Mit GROUP BY → eine Zeile **pro Gruppe**.
- UPDATE/DELETE ohne WHERE trifft **alle** Zeilen.
- Join über Komma + WHERE: Zeilen ohne Partner (Kunde ohne Bestellung) erscheinen nicht.
- OLTP = Row Store, Lesen und Schreiben, aktuell; OLAP = Column Store, Lesen, Historie.

DAS MUSST DU KÖNNEN

- ☐ Probleme des Dateiansatzes nennen
- ☐ DB, DBMS, Data Dictionary definieren
- ☐ Vor-/Nachteile DBMS nennen
- ☐ ER-Diagramm mit Kardinalitäten zeichnen
- ☐ Entität/Attribut/Schlüssel/Beziehung erklären
- ☐ relationale Begriffe (Tupel, Attribut, Domäne, PK, FK)
- ☐ ER → Tabellen (1:n, m:n)
- ☐ SELECT … FROM … WHERE lesen und schreiben
- ☐ Join, Aggregat, GROUP BY
- ☐ INSERT/UPDATE/DELETE
- ☐ OLTP vs. OLAP, Row vs. Column Store

ZUM SCHLUSS – STRATEGIE IN DER KLAUSUR

- Zuerst die Aufgaben mit sicheren Punkten (Nennen, MC, Skizzen), dann Rechnen. Pro Punkt etwa 1 Minute einplanen.
- Bei Rechnungen den **Rechenweg** hinschreiben (Restwertmethode, Invertieren +1, DNF-Terme) – das zeigt Verständnis, auch wenn am Ende ein Fehler steckt.
- Bei MC jede Option einzeln als wahr/falsch prüfen; nur ankreuzen, was du sicher als richtig erkennst.
- Skizzen groß, beschriftet, mit Pfeilen; Operator beachten („nennen“ = Stichpunkte, „erläutern“ = mehrere Sätze).

Spickzettel – alles auf einer Seite

Vorabend

Klausur

07.12.2026 · 48 Min · ca. 48–52 P → **1 P ≈ 1 Min**. Matrikelnr., nicht Name. MC: mind. 1 richtig, je Option ±½, nichts angekreuzt = 0. Nennen = Stichpunkte · kurz erläutern = 1–2 Sätze · erläutern = > 2 Sätze · skizzieren = beschriftete Zeichnung.

1 Information · Digitalisierung (3 P)

Daten + **Kontext** = Information → Wissen → Handlung (7 °C DE/Kanada/°F). Sichtweisen: **betriebswirtschaftlich** (neue Prozesse, Produkte, Geschäftsmodelle), **gesellschaftlich** (Lebenswelt digital, Disruption), **technisch** (analog → zeit- und wertediskret, binär). Phasen: Funktionen → Funktionsbereiche → integrierte Prozesse → 1:1 überbetrieblich → m:n bis Endkunde. Nutzen: Automatisierung → Informatisierung → Transformation.

2 IT-Infrastruktur · Cloud (4 P)

7 Komponenten: Hardware, Betriebssysteme, Datenverwaltung, Netzwerke/TK, Anwendungen, Internet als Plattform, Berater/Systemintegratoren. **5 Epochen** (überlagern sich): Groß-/Minirechner 1959 → PC 1981 → Client-Server 1983 → betriebl./internetbasiert 1992 → Cloud 2000. 2-Tier vs. N-Tier (Web → Anwendung → DB). Cloud = über das Internet beziehen, Abo, Browser; IaaS · PaaS · SaaS (Salesforce). AWS seit 2002. Metcalfe $n \cdot (n-1)$.

3 Moore (2 P)

1965: **Transistoren pro Chip** verdoppeln sich jährlich, später **alle 2 Jahre**. Interpretationen (nicht Moore): Leistung Mikroprozessor/Computer ×2 alle **18 Monate**, Preis halbiert sich alle 18 Monate. Fallen: verdreifacht, Speicherkapazität, Größe.

4 Zahlen (3 P)

2^n	1	2	4	8	16	32	64	128	256	1024

Hex A=10 … F=15. Dez → b: Restwert, **von unten lesen**. Bin → hex 4er-, oktal 3er-Gruppen **von rechts**. 235 = $1110\ 1011_2$ = 353_8 = EB_{16}.

Zweierkomplement (feste Bitbreite!): invertieren + 1. 8 Bit: −128 … +127, MSB zählt −128. −17 = 1110 1111. Einerkomplement/Vorz.-Betrag: −127 … 127, zwei Nullen. a − b = a + (−b), Übertrag vorne streichen. **Überlauf**: + + → − oder − − → +. 127 + 1 = 1000 0000 = −128.

Nachkomma: ×2, Vorkommabit notieren, **von oben lesen** ($0{,}375 = 0{,}011_2$; 0,4 periodisch). Gleitkomma $m \cdot b^e$; IEEE 754 32 Bit = 1 | 8 (Bias 127) | 23, Hidden Bit.

5 Logik (4 P)

AB	&	≥1	=1	NAND	NOR	XNOR
00	0	0	0	1	1	1
01	0	1	1	1	0	0
10	0	1	1	1	0	0
11	1	1	0	0	0	1

IEC: & · ≥1 · =1 · 1, Kreis = NICHT. **De Morgan** ¬(A∧B) = ¬A∨¬B, ¬(A∨B) = ¬A∧¬B. **DNF**: 2^n Zeilen; jede 1-Zeile ein UND-Term (1 → A, 0 → ¬A); mit ∨ verbinden. **Kernregel** (A∧B)∨(A∧¬B) = A. Absorption A∨(A∧B) = A. NAND universell: NOT A = NAND(A,A). Halbaddierer s = XOR, c = AND.

6 Programmierung (2 P)

Algorithmus = endlich; Programm darf endlos laufen; Prozess = laufendes Programm. Maschinensprache (prozessorabhängig) → Assembler → Hochsprache. Compiler vorab, Interpreter zur Laufzeit (Bytecode). Syntaxfehler meldet der Compiler, Semantikfehler nicht. Rekursion braucht Abbruchbedingung; int/int in Java ganzzahlig (10/3 = 3).

7 Datenstrukturen (6–8 P)

Array: feste Länge, Index ab 0, direkter Zugriff. Liste: nur Kopf bekannt, Zeiger umhängen. Doppelt: Vorgänger + Nachfolger. **Stack LIFO** push/pop/top (Tellerstapel). **Queue FIFO** put/get = enqueue/dequeue (Kasse). Fehlerfall: leer. **BST**: links kleiner, rechts größer; Löschen: Blatt weg · 1 Kind rückt nach · 2 Kinder → größter links/kleinster rechts. Voll = keine Halbblätter; sortiert eingefügt → degeneriert.

8 Suchen · Sortieren (6 P)

Verfahren	Voraussetzung	O
lineare Suche	keine	n
binäre Suche	sortiert, Array	log n
Bubble Sort	–	n^2
Merge Sort	– (Teile u. Herrsche)	n log n
Quicksort (Erg.)	Pivot	Ø n log n

Bubble: Größtes nach 1. Durchlauf hinten. Merge: Sortieren passiert beim Zusammenführen. 1000 Elemente → binär ≈ 10 Vergleiche.

9 Von Neumann vs. Harvard (8 P)

5 Komponenten: Rechenwerk (ALU), Steuerwerk, Bus, Speicherwerk, E/A-Werk. **Zyklus:** Fetch → Decode → Fetch Operands → Execute → Update Instruction Pointer. VN: **ein** Speicher, **ein** Bus → Flaschenhals, billig, flexibel, Buffer Overflow. Harvard: **getrennte** Speicher und Busse, gleichzeitig, schneller, teurer; Mikrocontroller, DSP, KI. Befehlszähler = Adresse nächster Befehl; Befehlsregister = aktueller Befehl. Steuer-/Adress-/Datenbus.

10 Speicher · RAID (4 P)

Register → Cache → RAM | Flash/SSD → HDD → Band: nach unten größer, langsamer, billiger; Strich = volatil | persistent. Cache wirkt durch **Lokalität**. n Adressleitungen → 2^n Byte (32 → 4 GB).

RAID	Prinzip	min.	Ausfall	Nutz
0	Striping	2	0	n·C
1	Spiegel	2	1	n·C/2
5	Parität	3	1	(n−1)·C
6	2× Parität	4	2	(n−2)·C

RAID ersetzt kein Backup.

11 Datenbanken · SQL (4–6 P)

Dateiansatz: Redundanz/Inkonsistenz, Programm-Daten-Abhängigkeit. DB + DBMS = Datenbanksystem; **Data Dictionary** = Metadaten. ER (Chen): Rechteck Entität, Raute Beziehung, Ellipse Attribut, unterstrichen Schlüssel; 1:1, 1:n (FS auf n-Seite), m:n (Hilfstabelle). Relation = Tabelle, Tupel = Zeile, Attribut = Spalte.

```
SELECT Spalten | AGG(...)
FROM   T1 JOIN T2 ON T1.FS = T2.PS
WHERE  Bedingung   -- 'Text', > vs >=
GROUP BY Spalte;   -- „pro …“
```

Ausführen: FROM → WHERE → GROUP BY → SELECT. Aggregat ohne GROUP BY = 1 Zeile. UPDATE/DELETE ohne WHERE = alle Zeilen. OLTP Row Store, lesen + schreiben · OLAP Column Store, lesen, Historie.